
Cogeto

A Verifiable, Sovereignty-First Memory Architecture for Large Language Model Assistants

*Design and mechanisms of a private, EU-hosted system for correctable long-term
memory, corpus-level contradiction detection, and provable deletion*

Ivan Golubić, bacc. ing. mech.

MVT Solutions Group d.o.o., Croatia, European Union

MCTO Advisory d.o.o., Croatia, European Union

info@mvt-solutions.com · hi@mctoadvisory.com · ivan@themrcto.com

COGETO · cogeto.eu · 29 July 2026

Abstract

Large language model assistants are increasingly entrusted with the working record of professional and industrial life: specifications, decisions, commitments, correspondence, and the document sets that regulators and customers hold organizations to. The memory layers attached to such assistants are typically opaque accumulators. They remember, but they cannot show what they remember, why they believe it, whether it is still true, whether two of their own sources disagree, or whether anything was ever really deleted. For privacy-sensitive users, and for organizations operating under the European Union's data protection and artificial intelligence regulations, that opacity is disqualifying. This paper describes Cogeto, a private, EU-hosted memory architecture in which every trust claim is backed by an inspectable artifact.

Cogeto models memory as discrete, source-linked atomic facts, each carrying a verbatim provenance span, an anchored subject, an explicit quality status from a closed vocabulary, a visibility scope, an orthogonal sensitivity flag, and a temporal validity interval. Facts enter memory only through a pipeline that reads every document format an organization actually holds, including scans, anchors each fact to the subject and revision of the document it came from, and submits every candidate to an independent verification pass against its own cited span; unsupported candidates are resolved automatically and recorded in an inspectable suppression log rather than queued for human triage. Stored facts are deduplicated, superseded, and checked for contradiction across the entire corpus by a reconciliation engine that resolves entities across aliases and languages, compares quantities and units arithmetically, and persists compatible verdicts so that judgments are stable over time. Retrieval fuses semantic, lexical, and entity signals under reciprocal rank fusion, treats visibility as a precondition evaluated inside the store rather than as a ranking signal, and analyses the resulting score distribution so that ambiguous questions fan out across the subjects the corpus actually holds and unanswerable ones are declared unanswered. Deletion is a compensating transaction protocol terminating in a signed, hash-chained receipt that a nightly sweep continuously re-verifies. Findings over a defined corpus are exported as a signed, printable report.

The architecture separates a synchronous fast path from an asynchronous slow path, isolates all model access behind a swappable gateway that defaults to a European provider and supports fully local models, and pseudonymizes sensitive entities locally before any external call. We describe the system's design principles, mechanisms, evaluation methodology, and regulatory alignment, and argue that verifiability, not capacity, is the property that makes machine memory trustworthy.

Keywords: large language models, long-term memory, retrieval-augmented generation, memory consolidation, contradiction detection, data sovereignty, right to erasure, verifiable systems, human oversight, hybrid retrieval, temporal data, regulated documentation

Contents

1. Introduction	4
2. Background and related work	6
3. Problem statement and design principles	8
4. A conceptual model of verifiable memory	10
5. System architecture	12
6. Reading and admission	14
7. Verification and automatic resolution	16
8. Reconciliation and consolidation	17
9. Temporal memory and point-in-time queries	20
10. Retrieval, ambiguity, and grounded answering	20
11. The findings report	22
12. Provable deletion	22
13. Human oversight as a state machine	24
14. The model layer	24
15. Trust artifacts beyond the core	26
16. Evaluation methodology and published results	27
17. Deployment, sovereignty, security, and operations	29
18. Discussion	31
19. Conclusion	32
Appendix A. Terminology	34
Appendix B. Operating parameters	36
References	37

1. Introduction

1.1 Motivation

The adoption of artificial intelligence in organizations has moved from experiment to default. The Stanford AI Index reports that 78 percent of organizations used AI in 2024, up from 55 percent a year earlier, with generative AI use in at least one business function more than doubling over the same period [40]. The trust required to make such systems genuinely useful has not kept pace with their capability. In a benchmark study of 2,600 privacy and security professionals, more than one in four organizations reported having banned generative AI tools at least temporarily over privacy and data security concerns, while nearly half admitted that non-public company information had been entered into such tools [41]. The tension is structural: the assistants that would be most valuable are precisely those that remember the most sensitive material, and the more they remember, the less their users are able to see, correct, or revoke.

Two populations feel this most acutely. The first is the independent professional and the small advisory team, whose working context is, in substance, other people's confidential information. Consultants, counsel, financial advisors, and fractional executives accumulate decisions, promises, and obligations across correspondence, calendars, and notes, and would be transformed by a system that could answer the question *what did I decide, promise, and commit to, and what is still open*. The second is the organization whose obligations are documentary. A medical device manufacturer maintains a technical file spanning specifications, risk analyses, verification reports, instructions for use, and change notices, accumulated over years and revised by many hands. An automotive supplier maintains requirement stacks against which every delivery is judged. An engineering firm maintains specification revisions whose disagreements surface, expensively, during acceptance. For these organizations the pressing question is not what a single document says. It is whether the document set is consistent with itself, which version of a contested statement is current, and how any answer can be evidenced to an auditor.

Neither population can submit that context to an opaque, foreign-operated service whose memory can be neither inspected nor provably erased. In the European Union this is not merely a preference. The General Data Protection Regulation establishes a right to erasure and mandates data protection by design and by default [34], with cumulative enforcement fines measured in the billions of euros [42], and the Artificial Intelligence Act introduces transparency obligations for AI systems [35]. The European Commission's data strategy explicitly frames control over data infrastructure as a matter of European sovereignty [36]. Sector regulation adds a second layer: the Medical Device Regulation requires manufacturers to maintain technical documentation that is complete and current [48], and the quality management standards under which such organizations operate require that records be controlled, traceable, and demonstrably maintained [49, 51].

We take this convergence of practical need and regulatory demand as a design brief. The question this paper addresses is not how to make an assistant remember more, but how to make what it remembers worthy of trust, and how to make the corpus it holds answer for its own consistency.

1.2 Position: from remembered to verifiable

Our central claim is that trust in machine memory cannot be asserted; it must be evidenced. A memory system earns trust when each of its trust claims corresponds to an artifact the user can inspect. We adopt this as an explicit design rule: if a trust claim has no artifact, it is marketing; if it has one, it is a mechanism. Concretely, in the system described here:

- The claim *I remember accurately* is evidenced by evaluation scores computed against a hand-labeled corpus, gated in continuous integration and published on every release, per language and per model configuration.
- The claim *I know where each fact came from* is evidenced by mandatory, non-nullable provenance carrying the verbatim source span, surfaced as a source link in the interface.
- The claim *I know when facts stop being true* is evidenced by temporal validity intervals and an explicit quality status on every fact, maintained by an autonomous consolidation process.
- The claim *I notice when my sources disagree* is evidenced by persistent contradiction relations carrying both claims, both spans, and the date the conflict was detected, and by their appearance in answers and in a signed report.

- The claim *I do not invent what I do not have* is evidenced by a deterministic analysis of retrieval results that declares silence rather than filling it, and by the visible separation of cited facts from model knowledge in every answer.
- The claim *I forget when told to* is evidenced by a signed, hash-chained deletion receipt produced by a compensating transaction protocol that removes the original bytes, the derived facts, and their vector representations, and then verifies the absence of remnants.
- The claim *I act only with permission* is evidenced by a server-side approval state machine whose every transition is audit-logged.
- The claim *your data does not leave your jurisdiction* is evidenced by single-tenant deployment on European infrastructure, a European primary model provider, local pseudonymization of sensitive entities before any external call, and the architectural ability to route all model traffic to fully isolated local models.

We refer to the resulting property as verifiable memory. It is, to our knowledge, a combination not offered by memory systems currently attached to assistants, which overwhelmingly optimize for capacity and recall rather than inspectability, correctability, corpus-level consistency, and provable forgetting.

1.3 Contributions

- A conceptual model of assistant memory as discrete, source-linked atomic facts with a closed status vocabulary, an orthogonal sensitivity axis, anchored subjects, explicit visibility scopes, and bi-temporal validity, designed so that correctness claims are checkable rather than assumed (Section 4).
- An architecture that enforces the model's invariants structurally: a modular monolith with domain-driven boundaries, a strict separation of a synchronous retrieval path from an asynchronous maintenance path, and a single source of truth from which all derived indices are rebuildable (Section 5).
- An admission pipeline that reads the formats organizations actually hold, including spreadsheets and scanned pages, anchors every extracted fact to the subject and revision of its document, and admits nothing that an independent verification pass cannot support against its own cited span (Sections 6 and 7).
- An automatic resolution policy that removes human triage from the critical path while preserving inspectability, through a suppression log that records every rejected candidate with its reason (Section 7).
- A reconciliation engine that detects contradiction across a whole corpus rather than within a document, resolving subjects across aliases and languages, comparing quantities and units arithmetically, and persisting compatible verdicts in a checked-pair ledger so that judgments do not drift with model variance (Section 8).
- A retrieval design that fuses semantic, lexical, and entity signals under reciprocal rank fusion, treats visibility and sensitivity as hard preconditions inside the store, and analyses the post-fusion score distribution to distinguish a confident answer, an ambiguous question, and a silent corpus without user configuration (Section 10).
- A signed findings report that turns corpus-level consistency into a forwardable artifact, scoped to a defined document selection and explicit about findings that cross its boundary (Section 11).
- A provable deletion protocol combining the saga pattern with hash-chained, signed receipts and autonomous remnant verification, giving operational form to the right to erasure (Section 12).
- A model access layer that is European by default and isolable by construction, with a local pseudonymization tier and full support for locally hosted models such that no content need ever leave the deployment boundary (Section 14).
- An evaluation methodology in which measured quality is a continuous integration gate and a published artifact, with per-language floors that prevent a weak language from hiding inside an aggregate (Section 16).

1.4 Scope of this paper

This paper describes architecture, mechanisms, and the regulatory context that motivates them. Commercial considerations are deliberately excluded. Where quantitative results appear, they are the figures published with a specific release of the system against the corpus described in Section 16, including the results that are unflattering, since a paper whose thesis is that trust claims require artifacts cannot exempt its own accuracy claims from that discipline.

2. Background and related work

2.1 Language models and retrieval augmentation

Modern assistants are built on Transformer-based language models [1] whose few-shot competence at scale was established by GPT-3 [2]. Such models are parametric: their knowledge is frozen at training time, is not attributable to sources, and cannot be selectively corrected or deleted. Retrieval-augmented generation addresses these limits by conditioning generation on documents retrieved at inference time [3], with dense passage retrieval providing the learned-embedding substrate [4] and retrieval-integrated pretraining demonstrating the value of making retrieval intrinsic to the model [5]. Surveys of the rapidly growing literature catalogue its variants and its central promise: grounding generation in inspectable, updatable external knowledge [6].

An adjacent line of evidence bears directly on our design. As context windows have grown, it has become tempting to treat a long window as a substitute for retrieval, placing whole document sets in front of the model. Empirical work shows that models use long contexts unevenly, with material in the middle of a long input recovered markedly less reliably than material at its edges [43]. This matters for the document-set problem: an approach that depends on the model attending to every pairwise combination of statements in several hundred documents has no coverage guarantee, no record of what was compared, and no reproducibility between runs. The architecture described here therefore performs comparison as an explicit, deterministic, recorded operation over a structured store rather than as an emergent property of a large prompt.

Cogeto is a retrieval-augmented system in the broad sense, but it inverts the usual emphasis. Most such work treats the external store as a passive corpus and concentrates innovation in retrieval and prompting. We treat the store itself as the primary object of engineering: what enters it, under what verification, with what metadata, how it is reconciled against what is already there, how it ages, and how it is provably removed.

2.2 Memory for language model agents

A distinct line of work equips agents with explicit memory. MemGPT frames the context window as a scarce resource managed like virtual memory, paging information between fast and slow tiers [9]. Generative agents demonstrated that long-horizon believable behavior requires a memory stream combined with periodic reflection, in which higher-level conclusions are synthesized from accumulated observations [10]. Recent surveys organize the design space along acquisition, management, and utilization dimensions [11]. These systems establish that memory management, not merely memory capacity, is the operative problem. Our work adopts that lesson and extends it in a direction the agent-memory literature has largely not pursued: memory as an auditable, correctable, legally accountable artifact with explicit quality states, corpus-level consistency checking, and provable deletion.

2.3 Hallucination, verification, and factual consistency

Language models generate fluent falsehoods. Surveys of hallucination in natural language generation [7] and in large language models specifically [8] document the phenomenon’s breadth and taxonomize its causes. Mitigations relevant to our design include Chain-of-Verification, in which a model drafts verification questions about its own output and answers them independently before finalizing [12]; SelfCheckGPT, which detects hallucination through sampling-based consistency without external resources [13]; and FActScore, which decomposes long-form generations into atomic facts and verifies each against a source [14]. The summarization literature contributes the complementary notion of factual consistency between a generated statement and its source document, together with model-based metrics for it [47] and meta-evaluations of those metrics across tasks [46]. Evaluation frameworks for retrieval-augmented systems have since formalized faithfulness and answer relevance as measurable properties of a pipeline rather than of a model [45].

Cogeto internalizes this literature at the point of memory formation. Extraction output is treated as a claim, not a fact, until an independently prompted verification pass confirms that the cited span supports it, and the atomic granularity of storage is chosen precisely so that verification has a well-defined object. The distinction we draw from the summarization line of work is one of position rather than technique: a consistency check applied at generation time protects one answer, whereas the same check applied at admission time protects every future answer that would have rested on the admitted fact.

2.4 Hybrid retrieval and rank fusion

Lexical retrieval in the probabilistic relevance framework, exemplified by BM25 [16], and approximate nearest-neighbor search over dense embeddings, exemplified by Hierarchical Navigable Small World graphs [17], have complementary failure modes: the former misses paraphrase, the latter misses exact identifiers, names, and codes. Reciprocal rank fusion is a simple, robust method for combining ranked lists from heterogeneous retrieval systems that outperforms more elaborate fusion and learning approaches in practice [15]. Cogeto's retrieval fuses three signals, semantic, lexical, and entity-based, under reciprocal rank fusion, and contributes a policy layer above the fusion: visibility and sensitivity operate as hard preconditions inside each underlying query, memory quality states operate as post-fusion score multipliers, and the shape of the fused score distribution is itself read as a signal about the question (Section 10).

2.5 Temporal data management

Facts are true during intervals, not forever. The temporal database literature distinguishes valid time, when a fact holds in the modeled world, from transaction time, when the database knew it, and this bi-temporal distinction was standardized in SQL: 2011 [18]. Cogeto applies bi-temporal modeling to assistant memory: every fact carries a validity interval maintained independently of its ingestion timestamp, superseded facts have their intervals closed rather than their rows destroyed, and point-in-time queries over past belief states are a first-class retrieval mode (Section 9).

2.6 Consolidation in biological memory

Our consolidation process borrows its name, dreaming, from a robust body of neuroscience. Sleep actively consolidates memory: newly encoded, labile traces are reactivated, redistributed, and integrated with existing knowledge during sleep, with sleep favoring the extraction of gist and the integration of new information into schemas [19, 20, 21]. Complementary learning systems theory explains why biological cognition separates a fast-learning hippocampal store from a slow-learning neocortical one: interleaved, offline replay allows integration without catastrophic interference [22]. Catastrophic forgetting, the tendency of connectionist systems to overwrite prior knowledge when trained sequentially, is the failure mode this separation avoids [23, 24], and it is one motivation for externalizing assistant memory into a non-parametric store rather than continually fine-tuning the model. We are explicit that the analogy is architectural, not biological: dreaming is a scheduled batch process, not a claim about machine cognition. But the division of labor it implements, fast online encoding followed by slow offline reorganization, deliberately mirrors the biological pattern, for the same reason: reorganization is expensive and must not block behavior.

2.7 Document understanding and optical character recognition

An organization's knowledge does not arrive in one format. Specifications arrive as text documents, test evidence as spreadsheets, older revisions as scans, and correspondence as mail with attachments. Mature open engines exist for converting page images into text, of which Tesseract is the canonical example [44], and their accuracy on clean printed material is high enough that an in-instance, processor-only deployment is practical, which matters when the material may not leave the boundary. Cogeto treats reading as a first-class stage with an explicit failure mode: material that cannot be read is labelled unreadable rather than silently reported as processed, because a document set that appears ingested but contributed nothing is worse than one that was never offered.

2.8 Distributed systems patterns

Cogeto's reliability mechanisms are deliberate applications of established patterns. Long-lived operations spanning multiple stores are structured as sagas, sequences of local transactions with compensating actions, following Garcia-Molina and Salem [25]; state changes and their downstream work are recorded atomically using the transactional outbox pattern [26]; the codebase is organized around domain-driven bounded contexts [27]; and the system is deliberately a modular monolith rather than a distributed microservice topology, consistent with the argument that monolith-first designs avoid premature distribution costs [28]. Tamper-evident audit structures follow the linked time-stamping construction of Haber and Stornetta [37] and the secure audit log design of Schneier and Kelsey [38].

2.9 Human oversight and privacy tooling

Guidelines for human-AI interaction emphasize efficient invocation, correction, and dismissal of machine initiative [29], and the human-in-the-loop literature surveys architectures in which human judgment gates machine action [30]. Cogeto operationalizes oversight as a server-side state machine rather than an interface convention (Section 13). For local privacy screening, mature open-source tooling exists: the Presidio framework provides configurable detection and anonymization of personally identifiable information [31], and GLiNER demonstrates that compact bidirectional transformer models can perform open-type named entity recognition efficiently on commodity processors [32]. European guidance on pseudonymization techniques [33] frames such screening as a recognized safeguard under data protection law.

2.10 Positioning

Table 1 summarizes how the strands above map onto the system's mechanisms; each mapping is developed in the section indicated.

Table 1. Literature and its correspondence to system mechanisms.

Literature	What it establishes	Cogeto mechanism	Section
Retrieval augmentation [3, 4, 5, 6]	External, updatable grounding beats parametric recall	The memory store as the engineered object; source-linked answers	4, 10
Long-context behavior [43]	Attention over very long inputs is uneven and unguaranteed	Comparison as explicit, recorded, deterministic operations over a store	8
Agent memory [9, 10, 11]	Management, not capacity, is the problem	Status vocabulary, consolidation, governance surface	4, 8
Hallucination and verification [7, 8, 12, 13, 14]	Model output requires independent checking	Verification pass at admission; measured agreement	7, 16
Factual consistency and evaluation [45, 46, 47]	Faithfulness to a source is measurable	Verification verdicts as gated, published metrics	7, 16
Rank fusion and hybrid retrieval [15, 16, 17]	Heterogeneous retrievers fuse robustly by rank	Three-signal retrieval under reciprocal rank fusion	10
Temporal data [18]	Facts hold during intervals; belief has history	Bi-temporal memories, point-in-time queries	4, 9
Consolidation neuroscience [19 to 24]	Offline reorganization is constitutive of memory	The dreaming cycle and its digest	8
Optical character recognition [44]	In-instance conversion of page images is practical	Local reading tier with an explicit unreadable state	6
Distributed patterns [25, 26, 27, 28]	Sagas, outboxes, bounded contexts, monolith-first	Deletion saga, transactional ingestion, module rules	5, 12
Oversight and privacy [29 to 33]	Human gating and local pseudonymization are practicable	Approval state machine; redaction tier	13, 14
Law, policy, and sector regulation [34, 35, 36, 48, 49, 51]	Erasure, transparency, sovereignty, and documentary control are obligations	Receipts, published artifacts, findings report, EU-first deployment	11, 12, 15, 17

3. Problem statement and design principles

3.1 The problem

We consider two related situations. In the first, a professional's context is distributed across correspondence, calendar, notes, and documents, and the assistant must answer questions about decisions, commitments, people, and open obligations across those sources. In the second, an organization holds a document set, hundreds of specifications, reports, manuals, change notices, and their revisions, and the pressing questions are whether the set is internally consistent, which statement is current where two disagree, and how any answer can be evidenced.

Both situations impose the same five requirements. The system must (i) answer from the material it holds; (ii) demonstrate, for any answer, where each supporting fact came from and how current it is; (iii) accept correction and propagate it; (iv) delete, on request, a source and everything derived from it, and prove that the deletion occurred; and (v) do all of the above without the material leaving infrastructure and jurisdictions the operator has chosen. The document-set situation adds a sixth: the system must compare statements it holds against one another and report where they conflict, with enough evidence attached that a reader can adjudicate without returning to the original files.

Requirements (ii) through (vi) disqualify designs that store raw text opaquely, mix users' data in shared indices, treat deletion as suppression, bind the system to a single foreign inference provider, or rely on a model attending to a very large prompt in place of an explicit comparison procedure.

3.2 Design principles

Seven principles govern the architecture. We state them here and refer back to them throughout.

- **P1, artifact-backed trust.** Every trust claim must correspond to an inspectable artifact: a receipt, a score, a source span, a status, an interval, a relation, an audit record, a report.
- **P2, facts, not documents.** The unit of memory is an extracted, verified, atomic fact. Raw documents are retained as originals for provenance, but they are never themselves the memory; unbounded raw text in the retrieval index is the primary cause of memory pollution.
- **P3, structural enforcement.** Invariants live in schemas, state machines, and module boundaries, not in conventions. A visibility rule that can be forgotten in one query is not a rule.
- **P4, fast path and slow path.** The user-facing path performs retrieval and answering only. Everything expensive, reading, extraction, verification, reconciliation, consolidation, deletion, reporting, runs asynchronously and never blocks a response.
- **P5, one source of truth.** The relational store owns all facts and metadata; every derived index, including the vector index, is rebuildable from it at any time. Nothing exists only in a derived store.
- **P6, sovereignty by construction.** Deployment is single-tenant on infrastructure of the operator's choosing; the primary model provider is European; sensitive entities are pseudonymized locally before any external call; and the model layer can be pointed entirely at local, isolated models without code change.
- **P7, the system resolves what it can measure, and only that.** Where an outcome can be judged against evidence, support, duplication, contradiction, staleness, ambiguity, the system decides and records its decision. Where an outcome depends on authority rather than evidence, who may see a document, which of two conflicting commitments the organization will honour, the system presents the choice with its evidence and does not simulate a judgment it cannot ground.

3.3 Non-goals

Stating what the system does not attempt clarifies what it does. Cogeto does not pursue personalization by model fine-tuning; user knowledge lives exclusively in the external store, for the reasons developed in Section 14.5. It does not adjudicate conflicting information on the user's behalf; contradiction is detected mechanically and resolved by the user, because on questions about an organization's own commitments the organization is the authority. It does not schedule or track work; open obligations are represented as facts with validity intervals and surfaced where the knowledge is, rather than as a task system with its own lifecycle. It does not present a review queue; candidates that fail verification are resolved automatically and recorded, because a memory system that generates homework is not used. It does not treat conversational fluency as a goal in itself; the conversational surface answers from retrieved, attributable facts, and an eloquent unsupported answer is scored as a failure, not a feature. And it does not claim that architecture alone constitutes regulatory compliance, which always retains organizational and contractual components; the claim, made precisely in Section 17, is that the architecture makes the technical share of compliance demonstrable.

4. A conceptual model of verifiable memory

4.1 The memory unit

The atom of the system is the memory: one durable fact, extracted from a source, carrying its complete trust metadata. A memory comprises an owner; a visibility scope; a sensitivity flag; a provenance reference including the verbatim span that supports it; an anchored subject; a quality status; a validity interval; the fact content; and a reference to its vector representation. Five of these fields are constrained in ways that carry the design.

Provenance is mandatory, non-nullable, and literal. Every memory references the exact source that produced it, an item of correspondence, a calendar event, a note, a document with its page, or the user's own statement in conversation, which is provenance too. Beyond the reference, the memory stores the *span*: the verbatim excerpt of the source that supports the claim. There is no such thing as an orphan fact. This constraint is what makes the interface's promise, click any fact and read the sentence it came from, universally true rather than usually true; it is what gives the verification pass of Section 7 a well-posed question; and it is what makes deletion cascades computable, because a source's derived facts are a closed, enumerable set.

The subject is anchored, not merely mentioned. A sentence in a datasheet may read only *the device has one antenna*. Stored as written, that fact is about a generic device and will collide with the corresponding sentence in the datasheet of a different model. Each source therefore carries a source context, derived once when the document is read, recording the subjects the document is about, its class, and its revision; extraction resolves each fact's subject against that context, preferring the nearest explicit subject in the text where a document covers several. Anchoring is the difference between a store that can compare facts and a store that merely holds them, and it is the precondition for both contradiction detection (Section 8) and ambiguity analysis (Section 10).

Scope is a closed enumeration, not free metadata. A memory is either private, visible only to its owner, or shared, visible to members of the same organization. Scope participates in every read path as a hard precondition (Section 10.2); it is a property the storage layer enforces, not a filter the application remembers to apply.

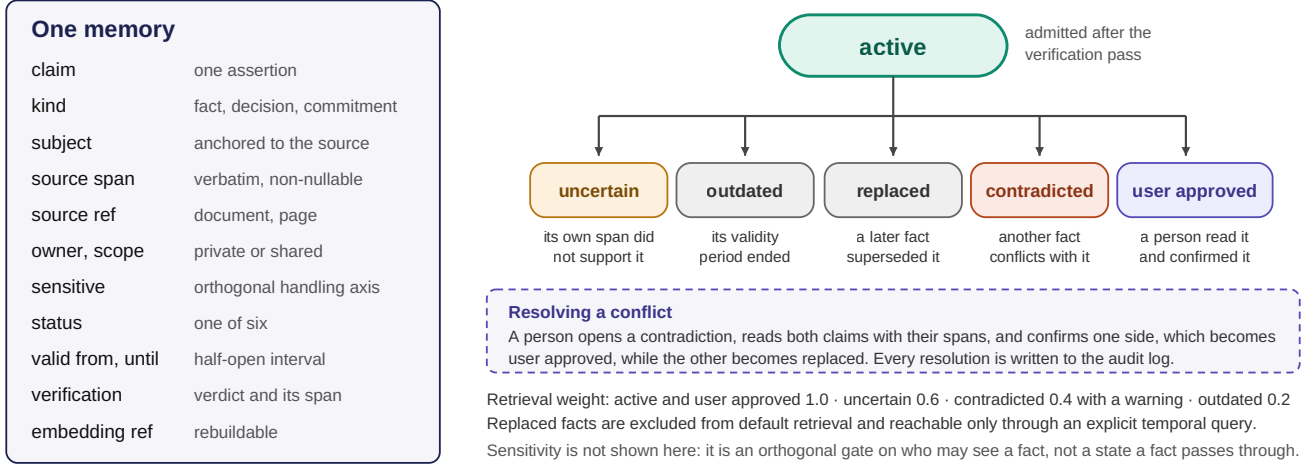
Sensitivity is an orthogonal axis, not a status. A fact may be shared in scope yet sensitive in handling, or private yet ordinary. Sensitivity governs how carefully a fact is surfaced and whether the original document may be opened, independently of who may see it and independently of how much it is trusted. Keeping the axes separate is what allows each to be simple enough to be provably ever-present.

Status is a closed six-state vocabulary. Every memory is in exactly one of the states in Table 2, and the state is part of the row, not an annotation. Transitions are owned: only reconciliation may mark a fact contradicted, only the user may mark it approved, only the deletion protocol may remove it. Ownership of transitions is what makes the states meaningful; a status anyone can set is a comment, not a state.

Table 2. The status vocabulary, its meaning, and the exclusive owner of each transition.

Status	Meaning	Who may set it
active	Current and verified; reached only after an independent check against the fact's own source span	The admission pipeline
user_approved	Explicitly confirmed by a person; the strongest state, never merged away by deduplication	Only the user
uncertain	Not adequately supported by its source, or hedged in it, or unjudgeable; retrievable but demoted and excluded from confident answers, and carrying the reason for its demotion	The verification pass or consolidation
contradicted	In conflict with another memory; both parties are marked and linked, with both spans and the detection date	Only reconciliation
outdated	Past its validity interval, or judged stale by consolidation	Consolidation or the user
replaced	Superseded by a newer fact, to which it points; excluded from default retrieval, preserved as history	Reconciliation, upon supersession or merge

Figure 1. The anatomy of a memory and every transition between its states. Each transition has exactly one owner and one cause.



4.2 Time as a first-class dimension

Following bi-temporal practice [18], every memory distinguishes when it was learned, the ingestion timestamp, from when it holds, the interval between its validity bounds. Supersession is an interval operation: when a newer fact replaces an older one, the older fact's interval is closed, its status becomes replaced, and it acquires a pointer to its successor. Nothing is destroyed. The result is that the store supports point-in-time queries, *what did we assert about this engagement in March*, and change queries, *what has changed on this account since the last review*, as retrieval modes rather than as reconstructions. We call the user-facing form of this capability time-travel memory: the assistant's beliefs have an undo history, and the history is itself inspectable.

4.3 Why a closed model

It is worth stating why the model is deliberately restrictive: mandatory provenance with a verbatim span, anchored subjects, closed scopes, a closed status vocabulary, owned transitions, preserved history. Each restriction converts a soft quality goal into a checkable invariant. *We know our sources* becomes a non-null constraint over a span. *We respect visibility* becomes a precondition no query can omit. *We handle conflicting information responsibly* becomes a pair of linked rows in a specific state, with both spans and a detection date, that a user can see and resolve. *We do not silently rewrite the past* becomes closed intervals and successor pointers. The model is the paper's first answer to its own question: memory becomes trustworthy when its correctness claims stop being prose and start being schema.

4.4 Scopes and collaborative memory

The two-scope model is minimal by intent, and the shared scope carries a specific collaboration thesis: small teams should collaborate through shared memory rather than through a shared conversation. When one member records or ingests material in the shared scope, a colleague's later question, *what did we decide on this account*, retrieves the shared facts with their full provenance, statuses, and history, without either party reading the other's private scope and without the team maintaining a single, ever-growing chat whose context no one governs. The design keeps the collaboration unit at the level where the trust machinery operates, the individual fact, so that everything this paper establishes about verifiability applies unchanged to collaborative knowledge.

Scope is a permission, not a judgment, and the system therefore never infers it from content. It is assigned from one of three sources, listed in Table 3: an explicit choice by a person present at the moment of ingestion, a rule a person wrote for a recurring flow, or the permissions of a source system that already encodes the answer. An item whose source permissions cannot be expressed in the two-scope model, a page restricted to a subset of a team, is not guessed at: it is skipped and reported in the synchronization summary, which is the conservative behavior and the honest one.

Table 3. Scope assignment by entry point. In no case is scope inferred from the content of a document.

Entry point	Rule	Source of authority
Deliberate upload	Explicit choice, defaulting to the user's setting	The person performing the upload
Capture in conversation	Private, stamped explicitly at capture	The system's default for personal statements
Correspondence intake	The recipient's default, refined by per-sender and per-alias routing rules	A rule written in advance by a person
Connected systems	Inherited: a team-readable container becomes shared, a personal or individually restricted item becomes that user's private context, a subset-restricted item is skipped and reported	The permissions of the source system

5. System architecture

5.1 A modular monolith with two processes

Cogeto is implemented as a modular monolith [28]: a single codebase organized into domain-driven bounded contexts [27], deployed as exactly two application processes. The application process serves the synchronous surface: the conversational interface, the governance surfaces, connector endpoints, and approval endpoints. The worker process executes everything asynchronous: reading, the ingestion pipeline, verification, reconciliation, consolidation, the deletion protocol, report generation, and the execution of approved actions. Both are built from one artifact and differ only in their endpoint.

The choice follows principle P3. The bounded contexts are code boundaries with mechanically enforced rules: each module exposes exactly one public interface; no module reads or writes another module's tables; cross-module communication travels as domain events through a transactional outbox; and job types are contracts with validated payloads rather than bare strings. A boundary, in this codebase, is the conjunction of imports, table ownership, job contracts, and dependency injection tokens, because a boundary enforced only at the import level is not enforced. Boundary rules are verified in continuous integration, since module boundaries that nothing enforces erode. At the same time, because the modules share one process and one database, the system avoids the operational and consistency costs of premature distribution [28], which matters doubly in a deployment model where every customer runs an isolated instance: each additional network service would be multiplied across every deployment.

Two of the modules are seams: thin leaf modules that isolate an external dependency behind a stable interface. The identity seam wraps the identity provider, which answers who a user is and what organization and roles they hold; crucially, the identity provider never decides which memories a user may see, since visibility is the memory model's own logic. The model gateway seam carries every model and embedding call in the system; no other module may communicate with a model provider directly. The gateway is the architectural basis of the sovereignty properties described in Section 14.

5.2 The two hard separations

Fast path versus slow path. Answering the user is synchronous and consists of exactly one kind of work: filtered, fused retrieval followed by generation. Reading, extraction, verification, deduplication, contradiction detection, consolidation, reporting, and deletion never execute inside a request. The separation mirrors the complementary-systems division discussed in Section 2.6: online behavior must be fast, and reorganization must be thorough, and the two requirements cannot be satisfied by the same process at the same time [22]. Practically, the separation is what allows the system to do far more work per memory, anchoring, verification, reconciliation, receipt generation, than a conventional memory layer without the user ever waiting on it.

Source of truth versus derived index. The relational database owns every fact and all metadata: memories, statuses, scopes, provenance, intervals, relations, receipts, audit records, evaluation records, and the job queue itself. The vector index stores embeddings together with a payload copy of exactly the fields needed for filtered search, and nothing else. It is rebuildable at any time by a single reindex operation that streams all memories from the relational store and regenerates the index. Nothing

exists only in the derived store. This asymmetry has three consequences that recur throughout the paper: disaster recovery reduces to restoring one database; embedding-model migration reduces to reindexing; and provable deletion becomes tractable, because there is exactly one authoritative record of what exists.

5.3 Transactional foundations

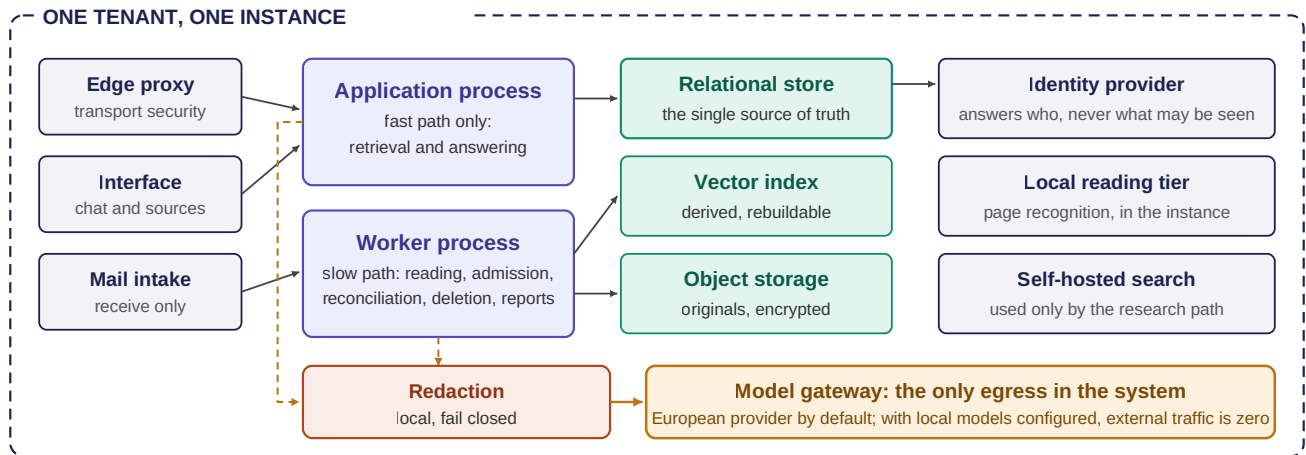
Every state change that must trigger downstream work, an item ingested, a document uploaded, a deletion requested, an action approved, a report commissioned, is recorded together with its job in a single database transaction, following the transactional outbox pattern [26]. The job queue itself is hosted in the relational database, so enqueueing is transactional by construction: it is impossible for the system to ingest an item and silently fail to process it, because the ingestion and the obligation to process it are one atomic write. Workers claim jobs with row-level locking; every job is idempotent under a deterministic key composed of the source type, the source identifier, and the job type; failures retry with exponential backoff; and jobs that exhaust their retries are parked in a dead-letter state visible in the governance surface rather than in a log file. A crashed worker loses nothing, and a duplicated delivery changes nothing.

5.4 Deployment unit

The deployment unit is one customer, one instance: an isolated composition of the two application processes, the relational database, the vector index, object storage for original files, the identity provider, a receive-only mail intake, a self-hosted search endpoint for the research path, a processor-only redaction sidecar, and an edge proxy, brought up by a single command on infrastructure of the operator's choosing. The idle footprint of the composition is approximately 1.3 gigabytes, which is what makes one instance per customer economically ordinary rather than exceptional. Original documents are stored encrypted at rest under keys scoped by organization, user, and visibility, and retrieved only through short-lived signed links.

Single-tenant isolation is itself a privacy mechanism: there is no shared index in which one tenant's embeddings neighbor another's, no cross-tenant cache, and no aggregate store whose access control must be trusted. The same composition, unchanged, is what a self-hosting operator runs, which makes the deployment model transparent rather than merely asserted.

Figure 2. The deployment unit. One image, two processes, one authoritative store, and a single model egress seam that every call passes through.



5.5 Interaction surfaces: conversation and proof

The architecture surfaces to the user through two deliberately distinct interfaces, and the distinction is itself a design claim.

The **conversational surface** is where knowledge enters and where questions are asked. The user asks in natural language, *what is the operating temperature range*, *what did I promise this client*, *what changed between revision B and revision D*, and the system answers from memory, drafts responses, and proposes actions that enter the approval flow of Section 13. Capture happens here too: a statement addressed to memory is extracted and verified like any other source, and a file attached in conversation is ingested through the normal pipeline by default, with the result confirmed in line, *added to*

sources: forty-seven facts, one contradiction. A file may also be marked transient, in which case it informs the conversation and is never stored. Concentrating capture in one place is a deliberate simplification: there is one door, and everything that passes through it is subject to the same admission rules.

The **sources surface** is where memory is inspected and proven rather than used. It has three levels. The first lists every source with its date, the number of facts derived from it, and badges for the contradictions, supersessions, and suppressions it is involved in. The second opens a source and shows every fact extracted from it, each with its status and the exact span it came from, together with the anchoring context, which is editable, and the log of everything the verification pass rejected from that document. The third opens a single fact and shows its entire life: its extraction, its verification verdict, the chain of facts that superseded it or that it superseded, and the answers that have cited it. A filtered search view spans all facts by status, by change date, and by content.

Two surfaces rather than one exists because of an observation we regard as a principle: a user cannot correct what a user cannot see. A memory system that offers only a conversational interface makes its store effectively write-only from the user's perspective, however good its answers; correctability, on which the entire status vocabulary depends, requires enumeration, and enumeration requires a browsing surface. The conversation is how memory is exercised; the sources surface is how it is governed and how its claims are proven; and neither is optional in the design.

5.6 Data and storage model

Three stores divide the system's data along lines of responsibility. The relational database holds the contractual core: the memory table, whose columns encode the model of Section 4 directly, an owner identifier and a scope enumeration, both non-nullable; a source type and source identifier, both non-nullable, together with the verbatim span; the anchored subject; the status enumeration; the sensitivity flag; the validity interval; the fact content and a reference to its embedding; and creation and update timestamps. Alongside it live the source table with its anchoring context and reading outcome; the relation table carrying contradiction and supersession links with their detection dates; the checked-pair ledger of Section 8; the suppression log of Section 7; the file metadata table; the deletion receipt table with its hash chain; the approval and audit tables; connector synchronization state with encrypted credentials; the prompt registry with versions and measured scores; the evaluation records; and the outbox and job tables. That the job queue, the audit trail, and the facts share one transactional store is not incidental economy: it is what makes the atomicity guarantees of Section 5.3 and the first step of the deletion saga of Section 12 possible as single transactions.

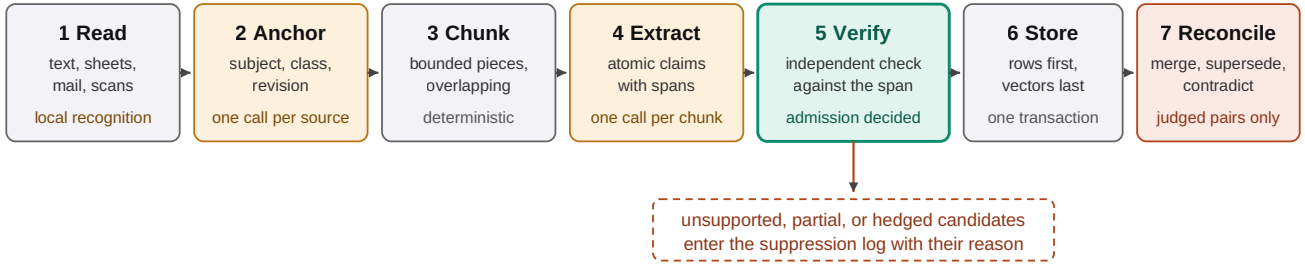
The vector index holds, for each memory, its embedding and a payload copy of exactly the fields that retrieval preconditions require, the owner, the scope, the sensitivity flag, the status, the source reference, and the validity horizon, with payload indices on the filterable fields so that the hard gates of Section 10 execute inside the index rather than after it. Object storage holds original document bytes, encrypted at rest, under keys structured as organization, then user, then scope, then a file identifier, a layout chosen so that the storage hierarchy mirrors the visibility model and so that keys remain stable across operational reorganizations. Access to originals is mediated exclusively by short-lived signed links, and the sensitivity flag gates whether such a link may be generated at all. The division of labor is summarized by principle P5: the relational store is the truth, the vector index is a rebuildable acceleration of it, and object storage is the provenance archive that makes source links literal.

6. Reading and admission

6.1 Pipeline structure

Content enters memory only through a seven-stage pipeline executed entirely on the slow path: read, anchor, chunk, extract, verify, embed and store, reconcile. Figure 3 shows the sequence, the model tier each stage uses, and the point at which the admission decision is taken.

Figure 3. The admission pipeline. Two stages are deterministic, four call a model, and one, verification, decides admission.



The gate before stage 4 decides whether a given source may produce facts at all, and at what budget; it is what keeps an observed channel from flooding the corpus.

6.2 Reading: the formats an organization actually holds

A memory system that reads only clean text quietly excludes most of an organization's record. The reading tier therefore covers text documents, correspondence with its attachments, spreadsheets, whose sheets and tables are flattened into extractable statements so that a test matrix becomes a set of claims rather than an opaque grid, and page images. Scanned material is converted in the instance by a processor-only recognition engine of the Tesseract class [44], with language packs for the languages the deployment uses. Where an instance runs a local vision model, difficult pages, poor scans, handwriting, dense tables, simple diagrams, are read through the vision tier of the model gateway instead. In both cases the recovered text enters the same pipeline as native text: it is anchored, extracted, verified against the recovered span, and given provenance to the file and the page.

The failure mode is explicit. A file that yields no readable text is marked unreadable on its source record and reported as such. It is never presented as processed, and no facts are produced for it. The distinction matters more than it may appear: a corpus that silently swallowed forty scanned pages looks complete and is not, and a system whose thesis is inspectability cannot afford a silent empty result.

Table 4. The reading tier, with the path each material takes and the honest outcome where reading fails.

Material	Path	Provenance recorded
Text documents and correspondence	Direct text extraction	Document, section, verbatim span
Spreadsheets and delimited data	Sheets and tables flattened into statements	Sheet, row and column region, span
Scanned or image-only pages	Processor-only recognition inside the instance	File, page, recovered span
Difficult pages where a local vision model is configured	Vision tier through the gateway	File, page, recovered span
Material that yields no readable text	Marked unreadable and reported	No facts produced

6.3 Anchoring: giving every fact a subject

Before chunking, one inexpensive model call reads the document's opening, its title block and headers, together with its filename, and produces the source context: the subjects the document concerns, which may be product models, project names, parties, or instruments; the document's class, such as specification, datasheet, manual, report, or contract; and its revision. Each element is marked confident or uncertain, because filenames lie and title blocks are sometimes absent.

The context is stored on the source and injected into every chunk's extraction call. Its effect is local and decisive: a sentence reading only *the device has one antenna* is extracted as a fact about the specific model the document describes. Where a document covers several subjects, section headings carry the per-section subject and the extractor is instructed to prefer the nearest explicit subject over the document default. Where the context is uncertain or multi-valued, extraction falls back to the subject as written, so that anchoring can only reduce ambiguity and never invent a subject that the document does not support.

Two consequences justify the extra call. First, without anchoring, near-identical boilerplate across a product family either merges into a single fact whose provenance now points at one arbitrary document, or is flagged as a contradiction between statements that were never in conflict; with anchoring, the same sentences become distinct facts about distinct subjects and neither error occurs. Second, anchoring is what gives the ambiguity analysis of Section 10.3 clean clusters to reason about. The context is visible on the source detail and editable; correcting it re-anchors that source's facts as supersessions, so a correction is itself part of the record rather than a silent rewrite.

6.4 The admission gate

Extraction is admission controlled per source and per connected system. The gate carries enable and disable, fact budgets, retention policy, and admission rules over channels, folders, senders, and document classes, and it is the reason a connected system that observes a high-volume channel cannot flood the corpus at full model cost. Material arriving from outside the organization additionally passes an injection defence before it reaches the extraction prompt, since a document is an untrusted input that will be shown to a model, and instructions embedded in it must not be mistaken for instructions from the operator.

6.5 Extraction and structured output

Extraction quality is determined as much by what the model is asked to produce as by the model itself, and the extraction prompt family is engineered around four commitments. **Structured output:** the extractor produces machine-parseable records with fixed fields, the claim, its kind, its subject, other entities, any attached condition, and any temporal anchors, validated against a schema at the gateway boundary, so that malformed output is a handled failure rather than a stored artifact. **Span anchoring:** every candidate must cite the source span that motivated it, which is what gives verification a well-posed question and gives the stored fact its literal provenance. **Relative-time resolution at extraction:** expressions such as *next Friday* or *after the budget is confirmed* are resolved, where resolvable, against the source's own timestamp into the validity fields of Section 4.2, because temporal anchoring performed later, without the source's context, is guesswork. **Calibrated abstention:** the prompt instructs, and the corpus of Section 16 tests, that a source containing nothing worth remembering yields nothing, since an extractor that always finds something is a pollution source, not a memory system.

Each commitment is embodied in a versioned prompt whose measured effect on the metrics of Section 16 is part of its release record, in line with the artifact discipline of Section 15.

6.6 Volume: importing a corpus

A memory system that begins empty delivers its value slowly, which is a practical problem when the material already exists as a folder of several hundred documents. Bulk import addresses this directly. A manifest is produced first, listing the files, their sizes and types, and removing exact duplicates by content hash before any model call is made. Ingestion is then queued through the ordinary pipeline under per-tenant concurrency limits, so that a large import degrades neither interactive answering nor another user's ingestion. Progress is visible while it runs, and completion produces a single summary of the form *four hundred and twelve documents, nine thousand eight hundred and forty-seven facts, thirty-seven contradictions, two hundred and fourteen superseded*. Every fact retains its pointer to the exact file, page, and span, so the summary is an index into evidence rather than a claim about it.

7. Verification and automatic resolution

7.1 Extraction output is a claim, not a fact

The literature summarized in Section 2.3 establishes that model output cannot be trusted by default, and that independent verification measurably improves factuality [12, 13, 14]. Cogeto internalizes this finding at the point where it matters most for a memory system: admission. Every candidate fact is submitted to a second model pass, prompted independently of the extractor, with a single question: does the cited source excerpt actually support this claim. The verifier receives the candidate and its span with a bounded window of surrounding context, and deliberately not the whole document, so that it judges evidence rather than repeating the extraction. It answers supported, partial, or unsupported.

A candidate becomes active if and only if the verdict is supported and the claim was not hedged in its source. Every other outcome is stored as uncertain, carrying the reason for its demotion, and is excluded from confident answers. A candidate omitted from a batched reply counts as unsupported: failure never defaults to acceptance.

Three design details matter. First, the verifier's prompt is disjoint from the extractor's, so the check is not the extractor grading its own work with its own rubric; this follows the logic of Chain-of-Verification, in which verification questions are answered independently of the original draft [12]. Second, verification operates at the granularity of the atomic fact, following the observation behind FActScore that long-form output must be decomposed before it can be meaningfully verified [14]. Third, verification outcomes are recorded and become part of the published evaluation of Section 16, so the mechanism's own accuracy is itself measured. The consequence for the memory model is that the status vocabulary is earned: a fact marked active has passed an independent check against its source, and the interface can say so without qualification.

7.2 Automatic resolution and the suppression log

An early form of this system queued every failed candidate for human review. The queue was correct in principle and unusable in practice: a few hundred documents produce hundreds of demoted candidates, and a person confronted with that list does not triage it, they abandon it, after which the queue's existence is worse than its absence because it implies a diligence nobody is performing.

The system therefore resolves these outcomes itself, under principle P7. Unsupported, partial, and hedged candidates are demoted automatically, never presented as confident knowledge, and never placed in a work queue. What replaces the queue is a record: every demotion writes a suppression log entry carrying the candidate, its source, its span, the reason, and the time. That log is queryable, appears on the detail view of the document it came from, and is summarized in the findings report of Section 11. Nothing is dropped silently, and nothing waits for a human.

The distinction between resolving and queueing is not merely ergonomic. It follows the division of principle P7: whether a span supports a claim is a question about evidence, which the system can and should decide; whether the organization will honour one of two conflicting commitments is a question about authority, which it must not. Verification outcomes fall on the first side of that line, and contradiction falls on the second, which is why the two are handled so differently.

A note on what is not automated. The system does not choose between conflicting sources, does not assign visibility from content, and does not delete anything it merely believes to be wrong. Each of these would substitute a model's judgment for a decision that belongs to the organization, and each is precisely the kind of silent action that makes a memory system impossible to audit. The system's autonomy is bounded by what it can evidence.

8. Reconciliation and consolidation

8.1 Why consolidation is necessary

A memory store that only accumulates degrades. Duplicates dilute retrieval; contradictions coexist silently; stale facts answer as if current; and the store's effective precision falls even as its recall grows. The agent-memory literature recognizes reflection and synthesis as necessary complements to accumulation [10, 11], and the biological literature is unequivocal that consolidation, the offline reorganization of recently encoded material, is constitutive of useful memory rather than an optimization of it [19, 20, 21]. Cogeto therefore treats consolidation as a first-class autonomous process with its own name, schedule, and user-facing artifact, and treats contradiction detection within it as a product capability rather than a housekeeping detail.

8.2 Candidate selection is deterministic

Reconciliation is expensive if performed naively: comparing every fact with every other fact is quadratic, and a corpus of ten thousand facts admits fifty million pairs. The design therefore separates candidate selection, which is deterministic, cheap, and exhaustive within its rules, from judgment, which is model-mediated and applied only to survivors. Candidates are drawn within the same owner and scope, ranked by embedding proximity, and bounded by a per-fact budget. Deduplication

candidates are pairs whose proximity is very high or whose entity overlap is high at identical kind. Contradiction candidates are pairs in a middle band of proximity, similar enough to be about the same thing and different enough not to be restatements, together with pairs that the deduplication judge has explicitly marked as related rather than identical, which is where paraphrased conflicts of very high similarity would otherwise escape.

Two properties of this arrangement deserve emphasis. Selection is recorded, so the system can state which pairs were compared, which is the difference between a coverage claim and a hope. And thresholds are calibrated for the embedding model in use and versioned alongside the reconciliation configuration, because a proximity constant means something different under a different embedder, and a threshold silently inherited across an embedding migration is a defect that presents as a mysterious drop in recall.

8.3 Subjects, quantities, and the two deterministic pre-steps

Two comparisons that models perform unreliably are performed before any model is consulted.

Subject resolution. Two facts about *Adriatic Foods* and *Jadranske hrane* concern the same company; two facts about the *gateway server* in different projects do not concern the same machine. Subject matching therefore resolves aliases, spelling variants, and cross-language names, and qualifies subjects that are local to a document's project or product with that context, while leaving genuinely global subjects unqualified so that conflicts across projects still pair. Where the resolver is uncertain it leaves the subject unqualified, which admits a pair that a person may dismiss, rather than qualifying it and hiding a conflict nobody will ever see. The asymmetry is deliberate: a false pair costs a reader a moment and is permanently dismissed; a missed pair is the failure the product exists to prevent.

Quantity comparison. A judge instructed to prefer compatibility when uncertain will not reliably notice that 3.2 millimetres and 3.4 millimetres are incompatible, or that fifty thousand and 50,000 are the same figure written differently. Quantities, units, and dates are therefore normalized and compared arithmetically before the pair is judged, and the comparison result is passed to the judge as evidence rather than left for it to infer. This is the least glamorous mechanism in the system and one of the most valuable, because in technical documentation most real conflicts are numeric.

8.4 Judgment, and the checked-pair ledger

Surviving pairs receive one model judgment each. Deduplication returns identical, distinct, or related; contradiction returns contradicts, compatible, or supersedes. The outcomes are shown in Figure 4.

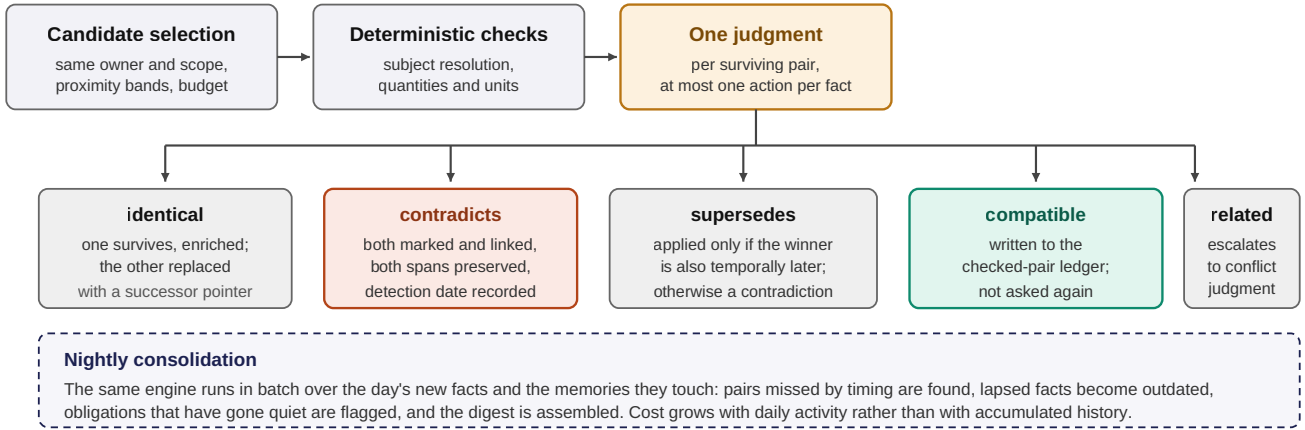
On **identical**, one fact survives, enriched where the duplicate carried additional detail, and the other becomes replaced with a successor pointer. Merging is conservative by design, because a false merge destroys a distinct fact, which is why the confirmation step exists and why merge decisions are among the evaluation harness's measured tasks, penalized asymmetrically.

On **contradicts**, both facts become contradicted and are linked by a permanent relation carrying both claims, both spans, both sources, and the date the conflict was detected. The system does not adjudicate: choosing a winner automatically would substitute the model's judgment for the organization's on precisely the question where the organization is authoritative. The conflict is surfaced where it is useful, on both documents, in any answer that cites either side, and in the report, and resolution, confirming one side, correcting both, or dismissing the conflict as a false pair, is a user action the audit log records.

On **supersedes**, the older fact's interval closes and it becomes replaced, but only if the proposed winner is also temporally later; where the model's verdict and the timeline disagree, the pair is downgraded to a human-facing contradiction. A model verdict alone never rewrites history.

On **compatible**, the pair is written to the **checked-pair ledger**. This is a small mechanism with three disproportionate effects. It makes judgments stable: a borderline pair judged compatible is not asked again on subsequent nights, so it cannot drift into a contradiction through sampling variance days after the documents last changed. It makes consolidation cheaper as the corpus grows, because the set of unjudged pairs shrinks toward the set of genuinely new ones. And it accumulates a labelled record of what the system compared and concluded, which is both an audit surface and the substrate for measuring the judge's own consistency over time.

Figure 4. Reconciliation. Deterministic selection and comparison precede any model call; each verdict has exactly one owned consequence.



8.5 The nightly cycle and its digest

Reconciliation runs twice: inline, when a fact is admitted, against the memories it touches, and nightly, in batch, over the day's admissions and their neighbourhoods. The second pass exists because the first is bounded. Two documents ingested within seconds of each other may not pair inline; a crowded topic may exhaust a per-fact budget before the true conflict is examined; a candidate demoted to uncertain does not pair until it is confirmed. The nightly pass closes those gaps, and because every relation carries the date the conflict was detected, a finding that appears later is distinguishable from one that was present from the start, which matters when the report is read by someone reconstructing a history.

Background intelligence that the user never sees builds no trust. Each cycle therefore produces a short morning digest, for example: four duplicate facts about a client were merged; a March pricing note now appears outdated in light of a newer message; two open obligations have gone quiet this week. Every line links to the exact memories, statuses, and sources involved, so the digest is not a summary of hidden work but an index into inspectable work. Nights in which nothing notable occurred produce no digest; a ritual that always fires becomes noise. The digest is where consolidation meets principle P1: the claim *the system maintains its memory responsibly* acquires a daily artifact.

8.6 On the analogy

We use the term dreaming advisedly. The analogy to sleep-dependent consolidation is architectural: like its biological counterpart, the process runs offline, reorganizes recently acquired material, integrates it with existing knowledge, extracts regularities, and resolves interference, and it exists for the same structural reason, that reorganization must not compete with real-time behavior [19, 22]. The analogy is not a claim about cognition, and nothing in the system's correctness depends on it. Its value is communicative: it names, accurately, the division of labor between a fast online path and a slow reorganizing one.

8.7 Cost characteristics

Consolidation is scheduled, incremental, and interruptible. Each cycle operates over the day's newly admitted facts and the existing memories they touch, rather than over the whole store, so its cost grows with daily activity rather than with accumulated history; periodic full passes exist but are infrequent and resumable. Because the process runs on the slow path under the job semantics of Section 5.3, every pass is idempotent and safely repeatable: an interrupted cycle resumes without double-merging or double-marking, which permits the process to run on modest, shared, and even preemptible compute. The checked-pair ledger bends the cost curve further, since the population of unjudged pairs shrinks over time even as the corpus grows. The property matters architecturally: consolidation quality is bought with off-peak batch time, the cheapest resource in the system, rather than with request-time latency or dedicated capacity, which is why the design can afford to be thorough.

9. Temporal memory and point-in-time queries

The bi-temporal model of Section 4.2 supports two retrieval modes beyond the default. A **point-in-time query** evaluates retrieval against the validity intervals as of a specified moment: *what did the documentation assert about this product in March* returns the facts whose intervals covered March, including facts since replaced, each labelled with its subsequent history. A **change query** computes the difference between two belief states: *what changed on this account since the last review* returns the facts whose status or interval changed in the window, each linked to the source that caused the change. In default retrieval, by contrast, replaced facts are excluded and outdated facts strongly demoted; temporal queries are the explicit, user-invoked mechanism that lifts those exclusions, and the mode is never inferred from the phrasing of a question, because a silent switch between belief states is precisely the kind of behavior that makes a memory system untrustworthy.

The combination yields what we call an undo history for machine belief: the assistant's past states are preserved, ordered, attributable, and queryable, and revising the present never falsifies the record of the past. We regard this property, more than raw recall, as what distinguishes memory from storage. It also carries direct practical weight in the documentary setting: the question *what did our documentation say when the customer placed this order* is answerable exactly, with citations, which is a question that ordinary document management answers only by manual archaeology through revision histories.

10. Retrieval, ambiguity, and grounded answering

10.1 Three signals, one fusion

A question fans out to three retrieval signals with complementary strengths: semantic search over the vector index [17], which captures paraphrase and topical similarity; lexical full-text search in the relational store, in the probabilistic relevance tradition [16], which captures exact terms, identifiers, part numbers, and amounts; and entity matching over the anchored subjects and named entities, which anchors questions that are fundamentally about someone or something. The three ranked lists are combined by reciprocal rank fusion [15], chosen for its robustness and its freedom from score-calibration assumptions across heterogeneous retrievers.

The question itself is first passed through one inexpensive rewriting call, which classifies it, resolves references against the conversation so that a follow-up such as *and the second one* becomes a well-formed query, produces the search string, extracts the entities to match, and selects the temporal mode. Explicit commands are matched deterministically before the model is consulted, so that a mechanical instruction is never at the mercy of a paraphrase. The rewriting step is load-bearing for quality and is measured as its own task in the evaluation harness, because a misresolved reference silently retrieves the wrong facts and the answer that follows will be confidently wrong about the wrong subject.

10.2 Gates before scores

The policy layer above the fusion embodies principle P3 and is, we would argue, the retrieval design's most consequential rule: **visibility is a precondition, never a preference**. Scope and the sensitivity flag are evaluated as hard filters inside each underlying query, as predicate clauses in the relational queries and as payload preconditions inside the vector search itself, so that an out-of-scope memory is never present in any candidate list at any stage. The alternative, retrieving broadly and filtering in application code, is rejected categorically: a demoted leak is still a leak, and a filter that lives in application logic is a filter that one future query will forget. Placing the gates inside the store makes the visibility rule non-bypassable by construction, and the equivalence of the two implementations, relational and vector, is covered by explicit leak invariants in the test suite.

Memory quality, by contrast, is a preference, and it is expressed as post-fusion score multipliers over the status vocabulary: active and approved facts carry full weight; uncertain facts are retrievable but marked; contradicted facts are retrievable with an explicit warning and both sources; outdated facts are strongly demoted; and replaced facts are excluded from default retrieval entirely, remaining reachable through the temporal queries of Section 9. The division is principled: who may see a fact is a rule, and how much a fact should be trusted is a judgment, and the architecture refuses to let the two share a mechanism.

10.3 Reading the shape of the result

Most retrieval systems treat the fused list as an answer to be summarized. Cogeto treats its *shape* as information. After fusion, the surviving results are grouped by anchored subject and the score distribution across those groups is examined. Three cases follow deterministically, with no user configuration and one behavior always.

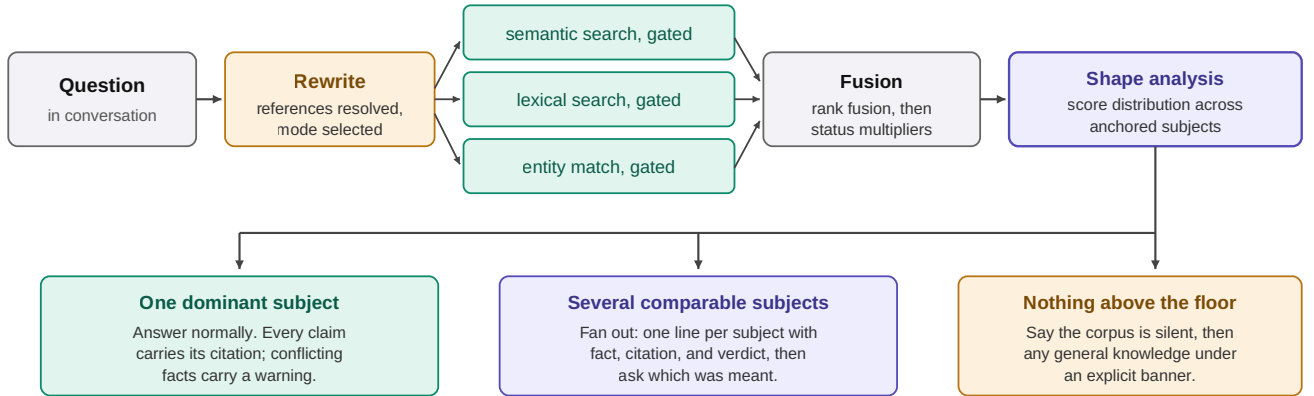
One dominant cluster. The question is about one thing the corpus knows; the answer proceeds normally, with citations.

No cluster above the relevance floor. The corpus is silent on the question. The answer says so explicitly, and may then continue with general model knowledge under a banner that marks it as not coming from the user's sources. This is a small mechanism with a large effect on trust: the most common way a document assistant misleads is not by inventing a citation but by answering a question its corpus never addressed, in the same confident register it uses for grounded answers.

Several comparable clusters with distinct subjects. The question is ambiguous with respect to what the corpus holds, which is common in a mixed corpus where an angle, a temperature, or a deadline exists in several unrelated documents. The answer then fans out: one line per cluster, each with its fact, its citation, and, where the question implies a judgment, the verdict for that subject, ending with a request to disambiguate. A silent guess and a bare request for clarification are both treated as defects, the first because it is wrong half the time and the second because it withholds an answer the system already has.

Cluster analysis is deterministic, costs no additional model call, and depends on the anchoring of Section 6.3 for clean groups, which is the clearest instance in the system of one mechanism paying for another.

Figure 5. The answering path. Gates are evaluated inside the stores, quality is applied after fusion, and the shape of the result decides the form of the answer.



Out-of-scope memories are absent from every candidate list at every stage, because the gate is a precondition inside the query rather than a filter applied afterwards. Replaced facts are excluded here and reachable only through an explicit temporal query.

10.4 Answer grounding

Retrieval terminates in generation, and the design constrains the hand-off. The generating model receives the fused, gated, weighted result set together with each fact's status and source reference, and the interface renders every claim in the answer with the status and source of the facts that support it. Claims that do not come from the corpus are marked as model knowledge rather than blended into the cited material. Facts in the uncertain or contradicted states, when they appear at all, appear visibly marked, so that the memory model's honesty about its own confidence survives into the conversational surface rather than being flattened by fluent synthesis. Where an answer cites a fact that is party to a contradiction, the conflict is stated in the answer itself, with both sources, rather than left for the user to discover.

The commitment is modest but strict: the system never presents a demoted fact as a current one, never presents model knowledge as corpus knowledge, and never presents any fact without the means to inspect it. There is no configuration that relaxes these rules, because a trust property that can be switched off is a trust property that must be re-established with every deployment.

11. The findings report

11.1 Why an artifact rather than a screen

Everything described so far is visible inside the system. That is sufficient for the person using it and insufficient for the situation that most often justifies its purchase, in which someone must demonstrate to a third party, an auditor, a notified body, a customer, a court, that a document set was examined and what was found. A conversation cannot be forwarded, filed, or signed. The report can.

The report is therefore treated as a first-class output of the architecture and not as an export convenience. It reuses the assembler and signing path already built for the passport and the deletion receipts, which means the guarantees are the same guarantees: a signature over a canonical structure, verifiable by a recipient who does not trust the instance that produced it.

11.2 Contents

The header states the instance, the corpus scope, the date range, the model configuration in use, and the published trust scores for that configuration, so that a reader knows both what was examined and how accurate the examining system measured itself to be. The body has three parts. **Contradictions:** each with both claims, both verbatim spans, each side's document with its revision and location, the date the conflict was detected, and its current resolution status. **Supersessions:** facts that have been replaced, with their chains, so that a reader can follow how a statement evolved and which revision closed it. **Suppressions:** a summary of the suppression log for the scope, stating what the verification pass rejected and why, which is what prevents the report from presenting a flattering picture assembled from convenient material.

Two formats are produced from one assembly: a paginated document for people and a structured export for machines, the latter so that findings can be tracked in whatever quality system the organization already runs rather than in ours.

11.3 Scope and boundaries

A report is always produced over an explicitly selected set of sources, never over whatever happens to be in the instance. The reason is practical: an instance may hold several projects, and the report that leaves the building for one customer must not enumerate another's material. Selection may be a folder, an import batch, a filter, or a workspace.

Scoping creates a boundary problem that the design addresses explicitly rather than by convention. A contradiction may have one side inside the selection and the other outside it, and both possible silent behaviors are wrong: including it exposes material beyond the scope, and omitting it hides a finding that may be the most consequential one, since conflicts between a project's documents and the organization's general standards are exactly the class nobody notices. Such findings therefore appear in a separately labelled boundary section, identified as referencing a document outside the audited scope, so that the reader knows what they are looking at and can request the wider examination if they wish. The header states the scope, so the signature covers something defined.

12. Provable deletion

12.1 From right to mechanism

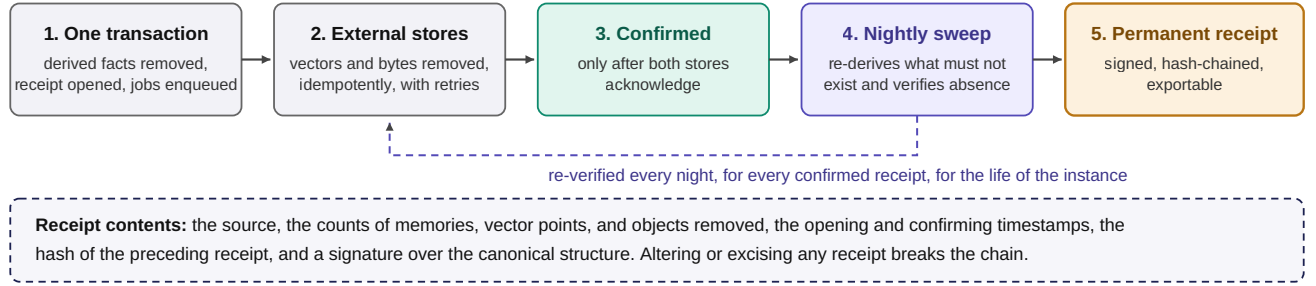
The right to erasure obliges controllers to delete personal data without undue delay on legitimate request [34]. In practice, deletion in AI memory systems is often suppression: the item stops appearing, while bytes, derived records, and index entries persist. Our position is that a deletion the user cannot verify is a deletion the user must take on faith, which contradicts principle P1. The system therefore implements deletion as a distributed protocol with a cryptographic artifact at the end.

12.2 The deletion saga

Deleting a document, or any source, spans three stores, the relational database, the vector index, and object storage, and therefore cannot be a single transaction. It is structured as a saga [25] in five steps, shown in Figure 6. First, a single relational transaction removes every memory derived from the source, removes the file metadata, writes a receipt row in a

pending state enumerating exactly what is to be removed, and enqueues the external deletions through the outbox; because provenance is mandatory, the set of derived memories is computable and complete. Second, the worker deletes the corresponding points from the vector index and the original bytes from object storage, idempotently and with retries. Third, the receipt transitions to confirmed only after both external stores acknowledge. Fourth, an autonomous nightly sweep verifies that no orphaned points or objects exist for any confirmed receipt, alerting on any discrepancy; the sweep is what upgrades the receipt from a record of intent to a continuously re-verified statement of fact. Fifth, the receipt becomes a permanent, user-visible artifact.

Figure 6. The deletion saga and its receipt. The nightly sweep re-verifies every confirmed receipt against the stores it claims to have cleared.



12.3 The deletion receipt

Each receipt records the source, the counts of memories, vector points, and objects removed, and the opening and confirming timestamps. Receipts are signed with an instance key and hash-chained, each incorporating the hash of its predecessor, following the linked time-stamping construction of Haber and Stornetta [37] and the tamper-evident audit log design of Schneier and Kelsey [38]: no receipt can later be altered or excised without breaking the chain. Receipts are permanent, exportable, and displayed in a dedicated section of the governance surface that we call, plainly, *Forgotten*. Receipt structures are backward compatible by rule, so that receipts issued by earlier versions of the system continue to verify after the system's own contents have changed, which is a small discipline with a large consequence: a chain that cannot be verified end to end proves nothing.

To our knowledge, a hash-chained, autonomously re-verified deletion receipt is not a feature of any widely deployed assistant memory system, and we consider it the clearest instance of principle P1: the claim *I forget when told to* reduced to an artifact that a compliance officer can file. An honest limitation accompanies the mechanism: receipts prove removal from the system's stores; they cannot retroactively unsend content already processed by an external model provider under that provider's retention terms. This is precisely why the deletion protocol is paired with the isolation options of Section 14, which bound what leaves the system in the first place.

12.4 A correctness argument

The protocol's guarantees can be stated informally but precisely. **Completeness of the cascade** follows from the provenance invariant: because every memory carries a non-nullable reference to its source, the set of records derived from a given source is a closed, computable query, and the saga's first transaction enumerates it exactly; there is no path by which a derived fact can exist outside the enumeration, because there is no path by which a fact enters storage without provenance. **Atomicity of intent** follows from the outbox: the removal of the relational records, the creation of the pending receipt, and the obligation to delete externally are one transaction, so the system cannot record a deletion it will not pursue. **Eventual completion** follows from the job semantics: the external deletions are idempotent and retried, so transient unavailability of the vector index or object store delays, but cannot lose, the obligation. And **detection of any residual divergence** follows from the nightly sweep, which re-derives, from the confirmed receipts, the set of identifiers that must not exist and verifies their absence in both external stores. The receipt chain then makes the whole history tamper-evident [37, 38]. None of these properties requires trusting the operator's diligence; each is carried by a mechanism, which is the section's point.

13. Human oversight as a state machine

The system drafts replies, proposes actions, and prepares reports, but consequential actions, sending a message, deleting data, writing to an external system, or modifying memory in bulk, execute only with explicit human approval. Following the human-oversight literature's emphasis on making machine initiative efficiently invocable, correctable, and dismissible [29, 30], the mechanism is a server-side state machine rather than an interface convention: a proposed action persists as a record moving through the states draft, pending approval, approved or rejected or expired, and executed. The approval endpoint is authenticated and does exactly one thing, transition the state; execution happens only in the worker, only from the approved state, and idempotently per action. A front-end confirmation dialog, however well designed, is explicitly insufficient, because a control that exists only in the client exists only until the first alternative client.

Every transition is written to the append-only audit log, so the answer to *what did the system do on my behalf, and who authorized it* is a query rather than an investigation. The same log records the resolutions a user applies to contradictions, the corrections applied to anchoring contexts, and every change of scope or sensitivity, which is what makes the memory's history complete rather than partially reconstructible: the store records not only what it believes but how its beliefs came to be revised, and by whom.

14. The model layer

14.1 The gateway seam

Every model interaction in the system, anchoring, extraction, verification, deduplication and contradiction judgment, query rewriting, drafting, vision reading, and embedding, passes through a single module whose interface is provider-neutral by requirement: it exposes completion, structured extraction, vision, and embedding operations in shapes that presume no particular vendor. No other module may address a model provider, and the prohibition is enforced by the same continuous-integration boundary checks that guard the other modules. The seam is a small amount of code with large consequences: it is the single point at which the system's entire relationship to model providers is defined, inspected, and changed, and the single place where redaction, budget accounting, and token measurement can be applied to every call without exception.

Model work is organized into tiers, and the tier is a property of the call site rather than a user preference. The pipeline tier serves the high-volume, bounded, structured tasks: anchoring, extraction, verification, reconciliation judgments, query rewriting. The answer tier serves user-facing synthesis. The embedding tier serves vectorization. The vision tier serves difficult page images. The separation is what allows the expensive tier to be reserved for the work a person actually reads, and it is also the unit at which localization decisions are made, since a tier can be moved to local infrastructure independently of the others.

14.2 Tier one: a European provider by default

The default configuration routes model traffic to Mistral AI, a Paris-based company that publishes open-weight models alongside its hosted interface [39]. The choice is deliberate on three grounds. First, jurisdiction: a European provider places the system's most sensitive data flow, model inference over extracted professional content, under European law and contractual frameworks, aligned with the sovereignty posture of the European data strategy [36]. Second, transparency culture: a provider that publishes open-weight models offers the ecosystem a degree of inspectability that pure interface vendors do not. Third, continuity with the isolation tiers below: because open-weight models from the same family can be run locally, the distance between the hosted default and a fully isolated deployment is operational rather than architectural. We phrase the data-handling point carefully: the provider offers European data residency options, and specific contractual terms are a matter for each deployment's data processing agreement rather than for this paper.

Provider and model configuration is data rather than deployment configuration. It lives in the database with encrypted credentials, managed through an administrative surface; only bootstrap values remain in the environment. The answering model is selectable by the user, while the models that decide what the system remembers, extraction and verification, are

administrative and are shown with the trust scores measured for that exact configuration, with untested combinations flagged as not evaluated. This is a direct consequence of Section 16's methodology: if scores belong to a configuration, then changing the configuration must show the user which scores now apply.

14.3 Tier two: local pseudonymization of sensitive entities

For deployments that want a hosted model's quality without exposing identifying details to any external party, the gateway applies a redaction tier: before any content leaves the instance, a local, processor-based named entity recognition stage detects sensitive entities, personal names, organizations, amounts, and configurable domain-specific categories, and replaces them with consistent pseudonyms; the response is re-identified locally before it reaches the user or the store. The stage is built on mature open tooling: the Presidio framework for configurable detection and anonymization [31] and compact open-type recognition models in the GLiNER family, which run efficiently on commodity processors [32]. Pseudonymization is a recognized safeguard in European data protection guidance [33], and its cost profile is decisive: roughly one gigabyte of memory, no accelerator, and milliseconds per document, so the privacy tier is effectively free at the scale of a single-tenant instance. The decorator fails closed: if redaction cannot run, the call does not proceed.

It is worth stating what the tier is and is not. It is a strong reduction of exposure for the entity categories it covers, not a guarantee that no sensitive information can leave in free text, which is why it composes with, rather than substitutes for, the isolation tier below.

14.4 Tier three: full isolation with local models

Because the gateway is provider-neutral, an instance can be configured so that no model traffic leaves it at all. Three local configurations are supported by construction. **Local embeddings:** embedding is the highest-volume model operation in the system, invoked for every stored fact and every query, and multilingual embedding models of modest size run acceptably on processors, so localizing embeddings removes the majority of external calls at negligible cost. **Local pipeline models:** the bounded, structured judgments of anchoring, verification, deduplication, and contradiction are within the competence of compact open-weight models served by a local runtime, and the gateway can route precisely these task classes locally while leaving user-facing generation hosted. **Full isolation:** every operation, including answering and vision reading, is served inside the deployment boundary, and external model traffic becomes zero. The three configurations are settings of the same seam, not variants of the system, which is the point: sovereignty is a property of the architecture, not of a product edition.

The honest trade-off is that local models are not free of quality consequences, and the arbiter is measurement rather than preference. Because trust scores are computed per configuration, an operator can see exactly what a given localization costs on each measured task and in each language, and the same harness governs the reverse direction: a task class migrates to a local model when, and only when, its measured quality clears the gates. This makes sovereignty a decision with a price tag attached rather than an ideological setting, which is the only form in which such a decision can be made responsibly.

14.5 Multilingual considerations

Users work in more than one language, often within a single document, and the model layer is specified accordingly. Extraction and verification prompts are evaluated per language against the corpus of Section 16, so that quality claims are never averages that conceal a weak language; the embedding model is selected for multilingual retrieval quality, since cross-lingual paraphrase, a commitment made in one language and queried in another, is a realistic case rather than an edge case; subject resolution in reconciliation crosses languages by design, since the same company or component appears under different names in different documents; and the pseudonymization tier's entity recognition is configured per language, since names, honorifics, and organization forms are language-specific.

Interface language and memory quality are deliberately distinguished. The interface is available in several languages, and adding one is a matter of key files and a synchronization check in the build. Memory quality in a language is a measured property that requires a labelled corpus in that language and its own gates. The trust surface states which languages are measured and which are interface-only, because conflating the two would be exactly the sort of quiet overstatement this architecture exists to prevent.

14.6 Externalized memory and the forgetting problem

A final motivation for this layer's design is negative: what the system deliberately does not do. Personalization by continual fine-tuning would entangle user knowledge with model parameters, making it uninspectable, uncorrectable, and undeletable, and would expose it to catastrophic interference, the well-documented tendency of neural networks trained sequentially to overwrite prior competence [23, 24]. It would also destroy every mechanism this paper describes: a fact absorbed into weights has no span to cite, no interval to close, no status to demote, no scope to enforce, and no receipt to issue when its owner asks for its removal.

By externalizing all user knowledge into the non-parametric, verifiable store, the system keeps the model stateless with respect to the user: any model, hosted or local, current or future, can serve any instance, and everything the system knows about a user remains in exactly one place, governed by exactly one set of mechanisms, including the deletion protocol of Section 12. The division we draw is between skill and knowledge. Skill, how to judge whether a span supports a claim, may legitimately live in weights, and a compact model specialized for the system's bounded judgment tasks is a reasonable object to train from labelled verdicts. Knowledge, what this organization asserts, may not. The customer's material is never training data, and that restriction is precisely what makes it possible to cite it, correct it, supersede it, and delete it with proof.

15. Trust artifacts beyond the core

Five further mechanisms extend principle P1 from the memory core to the system's periphery.

Prompts as versioned, published artifacts. Every prompt that decides what the system remembers, the anchoring, extraction, verification, deduplication, contradiction, and rewriting prompts, is versioned like a schema migration: numbered, immutable once released, and accompanied by a changelog. Because the system's core is published under an open licence, the prompts are public, and each released version carries its measured accuracy against the evaluation corpus. The user-facing meaning is unusual and, we believe, novel in this setting: here is the exact text that decides what we remember about you, and here is how well it measurably works.

The memory passport. A user can export everything, facts, statuses, provenance links, spans, validity history, relations, and deletion receipts, in a single, openly documented, versioned format. The export exists as an anti-lock-in commitment: a memory system whose value compounds over time acquires a natural hold over its user, and the honest response is to make leaving easy and staying a choice.

The trust score. The evaluation results of Section 16 are published per release, per language, and per model configuration, in the manner of a service status page. An accuracy figure that is public, historical, and attached to specific released versions is a fundamentally different object from an internal metric: it can be checked, cited, and held against the system.

Accounting and observability. Token consumption is recorded per operation at the gateway and rolled up per user, per instance, and per period, broken down by task family, which turns the question *what does this cost and where does it go* into a query. Where a comparison to the cost of not using the system is presented, its baseline methodology is stated and checkable rather than asserted as a multiplier, in keeping with the same discipline that governs accuracy claims. Operationally, the system exposes metrics per stage, provider, and job type, correlation identifiers that follow one request from interface through job to model call, and alerting on degraded health and integrity findings.

The open core. The system's source code is published under the GNU Affero General Public License, and the choice functions as a trust mechanism rather than a distribution strategy. Every enforcement claim this paper makes, that visibility gates live inside queries, that provenance is non-nullable, that the approval machine is server-side, that receipts are chained, is a claim about code, and publishing the code converts those claims from assertions into checkable statements. The licence's network provision, under which operators of modified network services must offer their modifications' source, extends the same reciprocity to the deployed form in which the system is actually consumed. Openness here is continuous with the paper's central discipline: a system whose thesis is inspectability should itself be inspectable.

16. Evaluation methodology and published results

16.1 The golden set

The system's quality claims rest on a hand-labeled corpus of realistic items, notes, correspondence, calendar entries, document excerpts, spreadsheet regions, and scanned pages, each annotated with the facts a diligent human assistant would extract, the subjects and conditions involved, expected temporal anchors, expected verification outcomes, and, for designed pairs, expected deduplication and contradiction relations. The corpus covers each supported memory language independently rather than in aggregate, includes deliberately hard cases, conditional commitments, relative dates, multi-fact sources, sources containing nothing worth remembering, near-duplicate pairs, and contradicting pairs, and grows permanently: every interesting real-world failure becomes a new case, reconstructed synthetically so that the corpus itself contains no real user data.

Three families of case have been added as the mechanisms they gate were introduced, and they are worth naming because each encodes a failure the system had to be designed against. **Anchoring cases** include two datasheets for different models of one product family that share boilerplate sentences verbatim; the requirement is that these neither merge into one fact nor raise a contradiction, and that a multi-model document anchors per section. **Quantity and cross-language pairs** include conflicts that are visible only arithmetically, three point two millimetres against three point four, and conflicts whose subjects are the same organization under names in two languages. **Ambiguity cases** include a fragment whose subject is resolvable only from the preceding conversation, a bare value that exists in several unrelated documents and must fan out across exactly the subjects that hold it, and a question the corpus does not address at all, which must be answered by declaring silence rather than by producing a plausible paragraph.

A vertical extension of the corpus, drawn from real regulatory and requirements documentation of the kind described in Section 1.1 and reconstructed so that it carries no customer material, exists for a specific reason: an engine measured only on notes and correspondence has not been measured on the document class whose consistency it is asked to certify. Matching between extracted and expected facts is semantic rather than literal, under a fixed and versioned similarity threshold, so that scores remain comparable across releases.

16.2 Metrics, gates, and floors

Measurements are computed on every change to prompts, models, or pipeline code, and each carries a threshold that operates as a continuous-integration gate: a change that regresses a metric below its gate does not ship, and thresholds may be raised but never lowered without a recorded decision, so that the trajectory of the thresholds is itself part of the auditable record. A metric drop beyond two points requires a written justification even when the result remains above its gate, which prevents a slow slide that never trips a threshold.

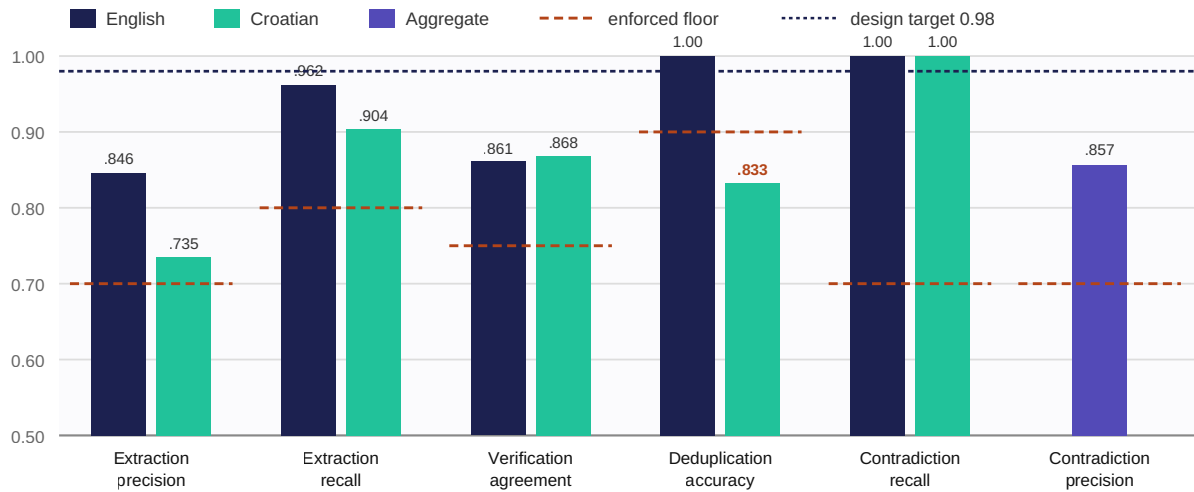
Floors and targets are different objects and the distinction is deliberate. A **floor** is the enforced minimum below which a change does not ship; it is set where the system measurably is, and it ratchets upward. The **target** is uniform at 0.98 across every measured task, and it is the level at which the system's claims about itself hold without qualification: at that accuracy an admitted fact is very unlikely to be unsupported, a detected conflict is very unlikely to be spurious, and a user reading an answer need not hold the measurement in mind. Publishing both, together with the current result, states plainly how far the system is from the standard it is built to meet rather than reporting only the threshold it happens to clear.

Three properties of the methodology carry most of its weight. **Floors are per language.** An aggregate over two languages can sit comfortably above a threshold while one language sits below it, and a system sold on the honesty of its numbers cannot permit that arithmetic. **Scores belong to a configuration.** Extraction and verification quality are properties of a specific model and prompt combination, so the published figures are keyed to that combination and an untested combination is labelled as untested rather than assumed to inherit. **Asymmetric penalties reflect asymmetric harms.** A false merge destroys a distinct fact and is penalized more heavily than a missed merge; a missed contradiction is worse than a false one, because a false pair costs a reader a moment and a missed pair defeats the purpose of the system.

Table 5. Measured tasks with their enforced floors, the design target the system is engineered toward, and the results published with the reference release. Figures are for the default hosted configuration.

Measured task	Floor	Target	English	Croatian	Reading
Extraction precision	0.70	0.98	0.846	0.735	Both languages sit well below target. Precision is weighted above recall at admission, because a stored falsehood is durable, so this is the principal quality work item and Croatian is the weaker column
Extraction recall	0.80	0.98	0.962	0.904	English is close to target; Croatian trails it. A missed commitment is the failure a user notices first, which is why the floor is high even though the target is higher still
Verification agreement	0.75	0.98	0.861	0.868	The admission check constrains everything downstream, so its distance from target is the most consequential single number in this table
Deduplication accuracy	0.90	0.98	1.000	0.833	English is at ceiling. Croatian is below the enforced floor, and the aggregate of 0.929 would have concealed that, which is exactly why the floor is per language
Contradiction recall	0.70	0.98	1.000	1.000	At target on the designed pairs of the corpus. That is a statement about the corpus, not a guarantee over all conflicts an arbitrary document set may contain
Contradiction precision	0.70	0.98	0.857 aggregate		Published because reporting recall alone would flatter the system. False pairs are cheap to dismiss, which is why precision carries a lower floor than the harm-weighted metrics
Grounded answering	every case	every case	27 of 27 hosted, 24 of 27 fully local		Answer cases are pass or fail rather than scored, and the gap between the two figures is the honest price of full isolation on this release

Figure 7. Published results for the reference release, shown against the enforced per-language floors and the design target of 0.98. The distance from target is published rather than presented as a gap to be closed quietly; the Croatian deduplication result is the case the per-language floor exists to expose.



Vertical axis truncated at 0.50 to make differences legible. Floors are enforced independently per language; the target applies to every task alike.

16.3 Publication and what it obliges

The figures above, per release, per language, per configuration, with corpus sizes and harness versions, constitute the published trust score of Section 15. We are aware that publishing one's error rates, and the distance between them and one's own target, is unusual; we consider it the evaluation methodology's point. Two consequences follow that are uncomfortable and therefore easy to omit.

First, a published result below its own floor, as in the Croatian deduplication figure above, becomes an obligation rather than an embarrassment to be managed: it is recorded, carries a decision, and is visible to any prospective user reading the same page a customer reads. Second, publication constrains what may be claimed elsewhere. A system whose extraction precision is measured at 0.789 in aggregate against a target of 0.98 cannot be described as extracting facts reliably without qualification, and the qualification belongs in the same sentence as the claim. This is the discipline the paper's title implies: verifiability is not only a property of the memory, it is a property of the statements made about the memory.

Evaluations run in continuous integration on every change, including on pull requests, where cached model responses make the gates fast and deterministic enough to run before a merge rather than after one. Retrieval and answering are measured as end-to-end cases rather than only by component, since a pipeline whose parts each pass their gates can still assemble an unsupported answer, and the case set includes the ambiguity and silence behaviors of Section 10.3 precisely because those are the behaviors under which a system is most tempted to appear helpful.

17. Deployment, sovereignty, security, and operations

17.1 Regulatory alignment

The deployment model of Section 5.4, one isolated instance per customer on infrastructure the operator chooses, composes with the mechanisms of the preceding sections into a specific regulatory posture.

Data protection by design and by default [34, Art. 25] is addressed structurally rather than by configuration: single-tenant isolation, mandatory provenance, hard visibility gates evaluated inside the stores, encryption at rest, local pseudonymization before external calls, and minimal external data flow are properties of the architecture. **The right to erasure** [34, Art. 17] is addressed by the deletion protocol and its receipts, which give the data subject not only the deletion but the evidence of it, and which a controller can file as part of its own accountability record. **Transparency expectations** of the kind the Artificial Intelligence Act introduces [35] are addressed by the system's disposition to show its work: a source span on every fact, a status on every belief, an audit record on every action, published prompts, and published accuracy. The broader ambition of **data sovereignty** [36] is addressed at the only level where it is meaningful, where bytes reside and where inference runs: on infrastructure the operator chooses, with a European provider by default and full isolation available by configuration.

A second regulatory dimension applies to the documentary use described in Section 1.1. Manufacturers under the Medical Device Regulation must maintain technical documentation that is complete, current, and internally consistent [48], and organizations operating quality management systems must keep records controlled and traceable [49, 51]. Nothing in this architecture performs a regulatory assessment or substitutes for one. What it provides is evidence of a specific kind: a signed statement of which documents were examined, which of their claims conflict, with the text of both sides, and what the examining system's measured accuracy was at the time. That is an input to a quality process, not a verdict from one, and the distinction is stated in the report itself.

We do not claim that architecture alone constitutes compliance, which is always also organizational and contractual. We claim that this architecture makes the technical share of compliance demonstrable rather than asserted.

17.2 Threat model and security posture

The security design is organized around four adversary classes, summarized in Table 6.

Table 6. Adversary classes and the structural response to each.

Adversary	Response	Residual risk
External network adversary	One hardened edge; transport security terminates at the proxy; every endpoint authenticated through the identity seam; originals reachable only through short-lived signed links whose issuance the sensitivity flag gates	Ordinary perimeter risk, bounded to one customer's instance
Curious or compromised co-tenant	There is no co-tenant. Isolation is per customer at the instance level, and no shared index, cache, or queue exists in which one tenant's data could neighbour another's	None of this class by construction
Adversary with read access to storage media	Data at rest encrypted in object storage and on host volumes; connector credentials additionally encrypted at the application layer	Key management remains an operational responsibility
The system's own future operators and code	Append-only audit log; hash-chained receipts; consequential actions only through the approval machine; secrets never in the repository or images; strict logging redaction so that memory content, document text, and credentials never appear in logs	A compromised host compromises its instance; the blast radius is one customer, which is the isolation model's purpose

Two further exposures deserve naming rather than burying. Documents are untrusted input that will be shown to a model, so instructions embedded in a document must not be able to act as instructions to the system; the admission gate of Section 6.4 applies injection defence to observed and third-party content for exactly this reason. And the external model tier of Section 14.2 is governed by contract and by the minimization tiers of Sections 14.3 and 14.4 rather than by the instance's own controls, which is why an operator whose threat model excludes external processing is offered a configuration in which no such processing occurs.

17.3 Operational reliability

The reliability posture follows from principle P5 and the job semantics of Section 5.3. Backup is one problem rather than three: the relational store, holding all facts, metadata, relations, receipts, and queue state, is dumped on a schedule to storage in a second location; original documents are replicated at the object-storage layer; and the vector index is deliberately not backed up at all, because the reindex operation reconstructs it from the source of truth, which turns the most awkward store to back up into the easiest one to recover. Restore is a rehearsed procedure rather than an assumption.

Failure handling is uniform: every background job is idempotent and retried with backoff; jobs that exhaust their retries are parked in a dead-letter state and surfaced in the governance surface, so that failure is a visible object an operator can inspect and re-run rather than a line in a log; and the nightly deletion sweep of Section 12.2 doubles as a continuous integrity check on the system's most sensitive invariant. Upgrades across a fleet of single-tenant instances use one versioned artifact per release, database migrations applied by a one-shot initialization step before the application starts, canary deployment to internal instances first, and rollback by artifact version plus, where the embedding model changed, a reindex.

Offline operation is a first-class configuration rather than a degraded mode. With local models configured, the instance requires no outbound connectivity: language and embedding models run locally, page recognition runs locally, search is self-hosted, storage is local, and there is no telemetry. The single model egress seam is what makes this verifiable rather than asserted, because there is exactly one place in the codebase where a call could leave, and the boundary check that guards it runs in continuous integration. For environments where even image distribution is unavailable, the instance ships as an offline bundle.

The design's operational thesis is the same as its trust thesis: fewer moving parts, one authority for state, and every failure mode terminating in an inspectable artifact rather than a silent divergence.

18. Discussion

18.1 An end-to-end illustration

A compressed scenario ties the mechanisms together. A manufacturer's quality lead points the system at the folder holding a product's technical documentation: several hundred files accumulated over four years, in two languages, comprising specifications through four revisions, risk analyses, verification reports as spreadsheets, instructions for use, change notices, and a set of older documents that exist only as scans.

The manifest runs first and removes a handful of exact duplicates by content hash. Reading follows: text documents directly, spreadsheets flattened into statements, scanned pages through the local recognition tier, three pages that yield nothing readable marked as such rather than passed over. Each document is anchored, so that a sentence in a revision C specification is a fact about that product at that revision rather than about a generic device. Extraction produces candidate facts, each carrying the exact sentence that supports it; verification admits those its span supports, demotes the rest, and records every demotion with its reason. Reconciliation runs, first inline and then overnight across the whole set.

The next morning the summary reads: several hundred documents, some thousands of facts, a few dozen contradictions, a few hundred superseded facts, and a suppression log. The contradictions are of recognizable kinds. The instructions for use state that the battery may be changed during operation, while the current specification requires re-initialization after any power interruption, because a change notice altered the power architecture and the manual was never updated. A verification report records a cold-start test at a temperature above the range the specification requires, which is not a versioning problem but a gap in evidence. Two interface documents disagree on a connector assignment, neither referencing the other, so supersession cannot resolve them and both are presented. A design review record retires an option that the current specification still lists.

Each finding carries both claims, both spans, both documents with their revisions, and the date it was detected. The quality lead works through them, confirming one side where the answer is clear, correcting where both are wrong, and dismissing the one pair that turned out to be two different components with similar names, each resolution recorded in the audit log. The signed report goes into the design review record, scoped to that product's documentation, with a labelled boundary section noting one conflict against a general standard held elsewhere in the instance.

Afterwards the system is used differently: engineers ask it questions. *What is the operating temperature range* returns the specification's figure with its citation and a note that the verification evidence covers a narrower range. *What changed between revision B and revision D* returns the superseded chain. *What did the documentation assert when this unit shipped* returns the belief state at that date. When a new change notice arrives by mail, it is ingested, and the chat confirms in line that it added twelve facts and conflicts with one interface document, which is the same finding that would previously have surfaced during an audit, arriving instead on the day it was created. Months later, when the customer engagement ends and the material must be removed, deletion runs as a saga and the receipt is filed.

Every step in the scenario is a mechanism from Sections 4 through 14, and every claim in it corresponds to something on a screen or in a signed file.

18.2 Why the design works

The mechanisms are individually modest: an enumeration, a non-null constraint, a second model call, a nightly batch job, an arithmetic comparison, a saga, a hash chain, a fused query, a state machine. None is exotic, and that is deliberate. The design's force comes from four disciplined combinations.

First, **the closed memory model converts quality goals into checkable invariants**, so that correctness claims are located in schema and state machines rather than in prose. Second, **the separation of paths buys the time to be careful**: anchoring, verification, reconciliation, and receipt generation are affordable precisely because they never block a response, which is how the system can do strictly more diligence per memory than an accumulator without feeling slower. Third, **determinism is placed where judgment is unreliable**: candidate selection, quantity comparison, subject resolution, gating, and cluster analysis are code, and the model is consulted only where a judgment about meaning is genuinely required, which bounds

both cost and variance. Fourth, **the artifact rule closes the loop between mechanism and trust**: each mechanism terminates in something a user, an auditor, or a partner can inspect, a span, a relation, a receipt, a digest, a score, a report, an export, so the system's trustworthiness is a property the user verifies rather than a property the operator asserts.

18.3 What is new

We claim novelty not for any component but for the assembly and for four commitments within it. The **admission-time independent verification pass**, with its outcomes feeding both the status vocabulary and a published score, moves hallucination mitigation from generation time, where the literature has concentrated, to memory formation time, where an error becomes durable. **Corpus-level contradiction detection with provenance**, resolving subjects across aliases and languages, comparing quantities arithmetically, and persisting its compatible verdicts so that judgments are stable, treats the consistency of a document set as a first-class product of a memory system rather than as a byproduct of question answering. The **hash-chained, autonomously re-verified deletion receipt** gives the right to erasure an operational artifact we have not encountered in deployed assistant memory systems. And the **publication of versioned prompts with measured, per-language, per-configuration accuracy**, stated against a uniform target rather than against a threshold chosen to be clearable, extends open-source transparency from code to the model-facing layer that actually determines the system's behavior.

18.4 Limitations

Several limitations deserve explicit statement, and they are properties of the design rather than defects awaiting a patch.

The verification pass reduces but cannot eliminate extraction error, and its own judgments are model-mediated; its measured agreement is published, and the mitigation is measurement rather than certainty. Contradiction recall is complete on the designed pairs of the evaluation corpus, which is a statement about the corpus rather than a guarantee over all possible conflicts in an arbitrary document set; conflicts whose incompatibility depends on domain reasoning the system does not perform, engineering implication rather than stated difference, are outside its reach. Quality is not uniform across languages, as Section 16 shows in the figures rather than in a caveat, and interface language support is deliberately distinguished from measured memory quality. Contradiction detection is conservative and deliberately non-adjudicating, which trades autonomy for correctness and leaves a review burden with the user, though the burden is now scoped to genuine conflicts rather than to every uncertain extraction. The pseudonymization tier covers configured entity categories and cannot guarantee that no sensitive information leaves in free text; the mitigation is the isolation tier. Deletion receipts prove removal from the system's stores and cannot retroactively affect content already processed externally under a provider's terms; the mitigation is bounding what is sent externally in the first place. The system knows only what was recorded, and no architecture recovers undocumented judgment. And the single-tenant model, while central to the privacy posture, multiplies operational cost per customer relative to pooled architectures; we regard the trade as the price of the isolation claims and have shaped the architecture, one artifact, two processes, few services, a small idle footprint, to keep that price low.

18.5 Future work

Three directions follow naturally from the design. **Richer temporal reasoning**: the bi-temporal store supports point-in-time and change queries, and the same substrate admits periodicity, expectation, and trend extraction, which would let the system observe that a commitment recurs or that a specification changes on a cadence. **Specialized judgment models**: the bounded tasks of verification, deduplication, and contradiction are narrow classification problems over short inputs, and the checked-pair ledger accumulates exactly the labelled material such a model would need; because scores are computed per configuration, migrating those tasks to a compact specialized model is a measured decision with a published price rather than a leap. **Interoperability of the passport format**: so that verifiable memory becomes a property users carry between systems rather than a property of any single system, which is the natural endpoint of an anti-lock-in commitment.

19. Conclusion

We have described Cogeto, a memory architecture for language model assistants organized around a single discipline: every claim the system makes about its own trustworthiness terminates in an artifact that someone outside the system can inspect. The discipline yields a closed, bi-temporal, provenance-mandatory memory model in which every fact carries the sentence

that produced it and the subject it is about; an admission pipeline that reads the material organizations actually hold and verifies each claim before believing it; automatic resolution of failed verification that leaves a record instead of a queue; reconciliation that treats the consistency of a whole corpus as its object, resolving subjects across languages and comparing quantities arithmetically before any model is asked; retrieval in which visibility is physics rather than policy and in which the shape of the result decides whether to answer, to fan out, or to declare silence; a signed findings report that turns internal consistency into something a third party can be handed; deletion that ends in a signed, chained, continuously re-verified receipt; oversight that is a state machine rather than a dialog box; and a model layer that is European by default and fully isolable by construction.

Individually, these are established techniques applied with care. Together, they constitute a position about what machine memory must become as assistants take custody of professional and industrial knowledge: not larger, but accountable. The value of an assistant is increasingly not what its model knows, since models are rented and replaceable, but what its memory holds and can prove. Most systems in this space remember. This one is built to remember responsibly, and to prove it.

Acknowledgments

The author thanks the collaborators of MVT Solutions Group d.o.o. and MCTO Advisory d.o.o. for discussions that shaped this work. Correspondence: ivan@themrcto.com; project information: cogeto.eu.

Appendix A. Terminology

The following closed vocabulary is used consistently throughout the system's code, schema, interfaces, and documentation; this paper uses it in the same senses.

Term	Definition
Memory	One stored, extracted, verified fact carrying owner, scope, sensitivity, provenance with its verbatim span, anchored subject, status, and validity interval; the system's atomic unit (Section 4.1)
Source span	The verbatim excerpt of a source that supports a claim, stored with the memory and used as the object of verification (Sections 4.1, 7.1)
Source context	The subjects, class, and revision derived once per document and injected into extraction, so that facts know what they are about (Section 6.3)
Scope	The visibility class of a memory, private or shared, enforced as a hard precondition in every read path (Sections 4.1, 10.2)
Sensitivity	An orthogonal handling axis governing how carefully a fact is surfaced and whether its original may be opened, independent of scope and status (Section 4.1)
Status	The memory quality state, one of six: active, user approved, uncertain, contradicted, outdated, replaced (Section 4.1)
Validity interval	The valid-time span during which a fact holds, maintained independently of its ingestion time (Section 4.2)
Supersession	The closing of an older fact's interval and its replacement by a successor, with history preserved (Section 4.2)
Admission pipeline	The seven-stage slow-path process: read, anchor, chunk, extract, verify, embed and store, reconcile (Section 6.1)
Verification pass	The independent model check that a candidate's cited span supports it, gating admission to the active state (Section 7.1)
Suppression log	The record of every candidate the verification pass demoted, with its span and reason, queryable and reported (Section 7.2)
Checked-pair ledger	The persisted record of pairs judged compatible, which stabilizes judgments and bounds recurring cost (Section 8.4)
Dreaming	The nightly consolidation cycle performing deduplication, contradiction detection, supersession, and staleness marking (Section 8.5)
Dreaming digest	The short morning artifact indexing the night's consolidation work, each line linked to its objects (Section 8.5)
Fast path, slow path	The synchronous retrieval and answering path versus the asynchronous maintenance path; the former never performs the latter's work (Section 5.2)
Hard gate	A visibility or sensitivity precondition evaluated inside the store's queries, never expressible as a score adjustment (Section 10.2)
Status multiplier	The post-fusion score factor applied per quality state in default retrieval (Section 10.2)
Shape analysis	The deterministic reading of the post-fusion score distribution across anchored subjects, deciding between a direct answer, a fan-out, and a declaration of silence (Section 10.3)
Findings report	The signed, scoped artifact stating contradictions with both spans, supersession chains, and the suppression summary (Section 11)
Deletion saga	The five-step compensating protocol that removes a source, its derived memories, and their vectors across all stores (Section 12.2)
Deletion receipt	The signed, hash-chained, autonomously re-verified record of a completed deletion, permanent and exportable (Section 12.3)
Approval state machine	The server-side lifecycle through which every consequential action passes (Section 13)
Model gateway	The single module through which all model and embedding calls pass; the seam at which providers are configured, localized, or fully isolated (Section 14.1)
Redaction tier	The local, processor-based pseudonymization of sensitive entities before any external model call (Section 14.3)
Golden set	The hand-labeled, per-language evaluation corpus against which every model-mediated task is continuously measured (Section 16.1)
Floor and target	The enforced minimum below which a change does not ship, and the uniform 0.98 standard the system is engineered toward; both are published alongside the current result (Section 16.2)

Trust score	The published, per-release, per-language, per-configuration evaluation results (Sections 15, 16.3)
Memory passport	The complete, openly documented export of a user's facts, statuses, provenance, history, relations, and receipts (Section 15)

Appendix B. Operating parameters

The following parameters govern the mechanisms described above. They are configuration in the sense that they have values, and contract in the sense that changing one requires a recorded decision and a re-run of the evaluation gates, since several of them are only meaningful relative to the embedding model in use.

Parameter	Value in force	Why it has this value
Chunk size and overlap	About 6,000 characters with 500 characters of overlap	Large enough for context, small enough for accurate extraction; the overlap prevents a claim split across a boundary from being lost
Verification window	The cited span plus about 240 characters	Enough context to judge the span, too little to re-extract from the document
Verification batch	Ten candidates per call	Reduces call volume by an order of magnitude on large documents without diluting the per-candidate question
Deduplication candidate	Very high proximity, or high entity overlap at identical kind	Duplicates are near-identical restatements; the second criterion catches restatements that embed differently
Contradiction candidate	A middle proximity band, plus pairs judged related by deduplication	Similar enough to be about the same thing, different enough not to be a restatement; the second clause closes the high-similarity paraphrase gap
Per-fact check budget	Three deduplication and three contradiction checks per pass	Bounds inline cost; the nightly pass and the ledger recover what the budget defers
Actions per fact per pass	One	Prevents a single admission from cascading through the corpus in one pass
Rank fusion constant	60	The conventional value from the rank fusion literature [15], retained because it is robust across heterogeneous retrievers
Status multipliers	1.0 active and approved, 0.6 uncertain, 0.4 contradicted, 0.2 outdated, 0 replaced	Quality is a preference expressed after fusion; visibility is a precondition expressed before it
Temporal lift	0.9 for outdated and replaced in the previous-belief mode	Makes superseded material first-class in an explicitly historical query and only there
Import concurrency	Capped per tenant	A large import must not starve interactive answering or another user's ingestion
Evaluation target	0.98 on every measured task	The level at which the system's claims hold without qualification; floors are the enforced minimum today and ratchet toward it
Metric justification threshold	Two points	Catches a slow decline that never trips a gate

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pp. 5998-6008. arXiv:1706.03762.
- [2] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 1877-1901. arXiv:2005.14165.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuettler, H., Lewis, M., Yih, W., Rocktaeschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pp. 9459-9474. arXiv:2005.11401.
- [4] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of EMNLP 2020*, pp. 6769-6781. doi:10.18653/v1/2020.emnlp-main.550.
- [5] Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M.-W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, PMLR vol. 119. arXiv:2002.08909.
- [6] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., and Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997.
- [7] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys* 55(12), Article 248, pp. 1-38. doi:10.1145/3571730.
- [8] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. (2025). A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems* 43(2), pp. 1-55. doi:10.1145/3703155. arXiv:2311.05232.
- [9] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560.
- [10] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*, Article 2, pp. 1-22. doi:10.1145/3586183.3606763.
- [11] Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., and Wen, J.-R. (2024). A Survey on the Memory Mechanism of Large Language Model based Agents. arXiv preprint arXiv:2404.13501.
- [12] Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., and Weston, J. (2024). Chain-of-Verification Reduces Hallucination in Large Language Models. *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563-3578. arXiv: 2309.11495.
- [13] Manakul, P., Liusie, A., and Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of EMNLP 2023*, pp. 9004-9017. doi:10.18653/v1/2023.emnlp-main.557.
- [14] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P. W., Iyyer, M., Zettlemoyer, L., and Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*, pp. 12076-12100. doi: 10.18653/v1/2023.emnlp-main.741.
- [15] Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2009)*, pp. 758-759. doi:10.1145/1571941.1572114.
- [16] Robertson, S., and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4), pp. 333-389. doi:10.1561/15000000019.
- [17] Malkov, Yu. A., and Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42(4), pp. 824-836. doi:10.1109/TPAMI. 2018.2889473. arXiv:1603.09320.
- [18] Kulkarni, K., and Michels, J.-E. (2012). Temporal Features in SQL:2011. *ACM SIGMOD Record* 41(3), pp. 34-43. doi: 10.1145/2380776.2380786.
- [19] Diekelmann, S., and Born, J. (2010). The Memory Function of Sleep. *Nature Reviews Neuroscience* 11(2), pp. 114-126. doi:10.1038/ nrn2762.
- [20] Stickgold, R. (2005). Sleep-dependent Memory Consolidation. *Nature* 437(7063), pp. 1272-1278. doi:10.1038/nature04286.
- [21] Rasch, B., and Born, J. (2013). About Sleep's Role in Memory. *Physiological Reviews* 93(2), pp. 681-766. doi:10.1152/physrev. 00032.2012.
- [22] McClelland, J. L., McNaughton, B. L., and O'Reilly, R. C. (1995). Why There Are Complementary Learning Systems in the Hippocampus and Neocortex: Insights from the Successes and Failures of Connectionist Models of Learning and Memory. *Psychological Review* 102(3), pp. 419-457. doi:10.1037/0033-295X.102.3.419.

- [23] French, R. M. (1999). Catastrophic Forgetting in Connectionist Networks. *Trends in Cognitive Sciences* 3(4), pp. 128-135. doi:10.1016/S1364-6613(99)01294-2.
- [24] McCloskey, M., and Cohen, N. J. (1989). Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In Bower, G. H. (ed.), *The Psychology of Learning and Motivation*, Vol. 24, pp. 109-165. Academic Press.
- [25] Garcia-Molina, H., and Salem, K. (1987). Sagas. *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pp. 249-259. doi:10.1145/38713.38742.
- [26] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island, NY. ISBN 978-1-61729-454-9.
- [27] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, Boston. ISBN 978-0-321-12521-5.
- [28] Fowler, M. (2015). MonolithFirst. martinowler.com, 3 June 2015. <https://martinfowler.com/bliki/MonolithFirst.html>.
- [29] Amershi, S., Weld, D., Vorvoreanu, M., Fournay, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., and Horvitz, E. (2019). Guidelines for Human-AI Interaction. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1-13. doi:10.1145/3290605.3300233.
- [30] Wu, X., Xiao, L., Sun, Y., Zhang, J., Ma, T., and He, L. (2022). A Survey of Human-in-the-loop for Machine Learning. *Future Generation Computer Systems* 135, pp. 364-381. doi:10.1016/j.future.2022.05.014.
- [31] Microsoft Presidio: Data Protection and De-identification SDK. Open-source software and documentation. <https://microsoft.github.io/presidio>.
- [32] Zaratianna, U., Tomeh, N., Holat, P., and Charnois, T. (2024). GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer. *Proceedings of the 2024 Conference of the North American Chapter of the ACL: Human Language Technologies (Volume 1: Long Papers)*, pp. 5364-5376. doi:10.18653/v1/2024.naacl-long.300.
- [33] ENISA, European Union Agency for Cybersecurity (2019). *Pseudonymisation Techniques and Best Practices: Recommendations on Shaping Technology According to Data Protection and Privacy Provisions*. November 2019.
- [34] Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 4 May 2016, pp. 1-88.
- [35] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, OJ L, 2024/1689, 12 July 2024.
- [36] European Commission (2020). *A European Strategy for Data*. Communication COM(2020) 66 final, Brussels, 19 February 2020.
- [37] Haber, S., and Stornetta, W. S. (1991). How to Time-Stamp a Digital Document. *Journal of Cryptology* 3(2), pp. 99-111. doi:10.1007/BF00196791.
- [38] Schneier, B., and Kelsey, J. (1999). Secure Audit Logs to Support Computer Forensics. *ACM Transactions on Information and System Security* 2(2), pp. 159-176. doi:10.1145/317087.317089.
- [39] Mistral AI. Company and model information. <https://mistral.ai/about>.
- [40] Stanford Institute for Human-Centered Artificial Intelligence (2025). *The 2025 AI Index Report*, 8th edition, Economy chapter. Stanford University. <https://hai.stanford.edu/ai-index/2025-ai-index-report>.
- [41] Cisco (2024). *2024 Data Privacy Benchmark Study*. Cisco Systems, San Jose, January 2024.
- [42] DLA Piper (2025). *GDPR Fines and Data Breach Survey: January 2025*, 7th annual edition. DLA Piper LLP.
- [43] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12, pp. 157-173. doi:10.1162/tac_l_a_00638.
- [44] Smith, R. (2007). An Overview of the Tesseract OCR Engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, pp. 629-633. doi:10.1109/ICDAR.2007.4376991.
- [45] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *Proceedings of the 18th Conference of the European Chapter of the ACL: System Demonstrations*, pp. 150-158.
- [46] Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kukliansky, D., Cohen, V., Scialom, T., Szpektor, I., Hassidim, A., and Matias, Y. (2022). TRUE: Re-evaluating Factual Consistency Evaluation. *Proceedings of the 2022 Conference of the North American Chapter of the ACL: Human Language Technologies*, pp. 3905-3920. doi:10.18653/v1/2022.naacl-main.287.
- [47] Kryscinski, W., McCann, B., Xiong, C., and Socher, R. (2020). Evaluating the Factual Consistency of Abstractive Text Summarization. *Proceedings of EMNLP 2020*, pp. 9332-9346. doi:10.18653/v1/2020.emnlp-main.750.
- [48] Regulation (EU) 2017/745 of the European Parliament and of the Council of 5 April 2017 on medical devices. *Official Journal of the European Union*, L 117, 5 May 2017, pp. 1-175.
- [49] ISO 9001:2015. *Quality management systems, Requirements*. International Organization for Standardization, Geneva.
- [50] ISO/IEC 27001:2022. *Information security, cybersecurity and privacy protection, Information security management systems, Requirements*. International Organization for Standardization, Geneva.

- [51] EN ISO 13485:2016. *Medical devices, Quality management systems, Requirements for regulatory purposes*. International Organization for Standardization, Geneva.